

Procesamiento de imágenes para visión artificial

2019-I

Carlos Alejandro Trujillo Anaya – carlostrujillo@itm.edu.co

Oficina M212

Departamento de electrónica y telecomunicaciones

Asignatura perteneciente a Ingeniería electrónica

Viernes 18-20

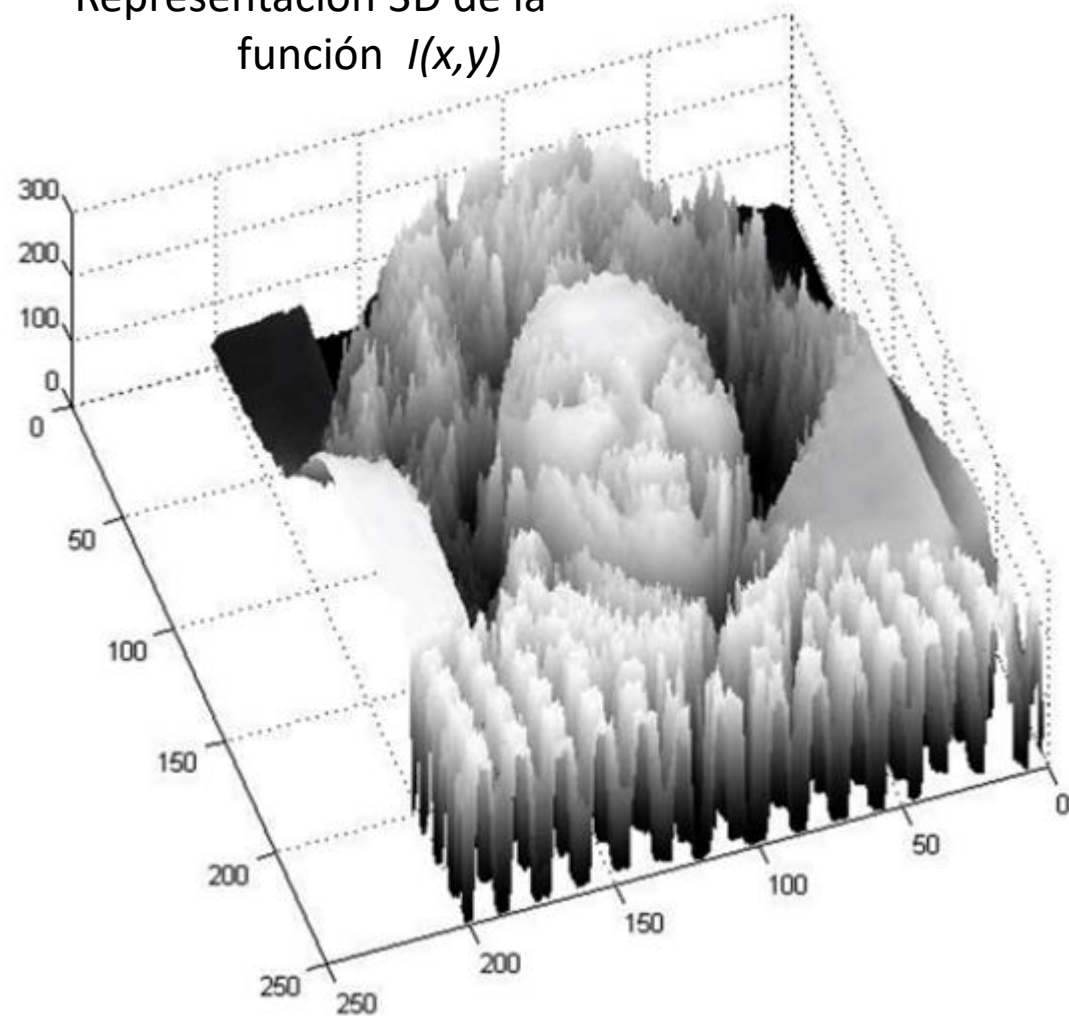
Aula O2



Esta imagen a blanco y negro
puede entenderse como una
función:

$$I(x,y)$$
$$f: R \times R \rightarrow R$$

Representación 3D de la
función $I(x,y)$

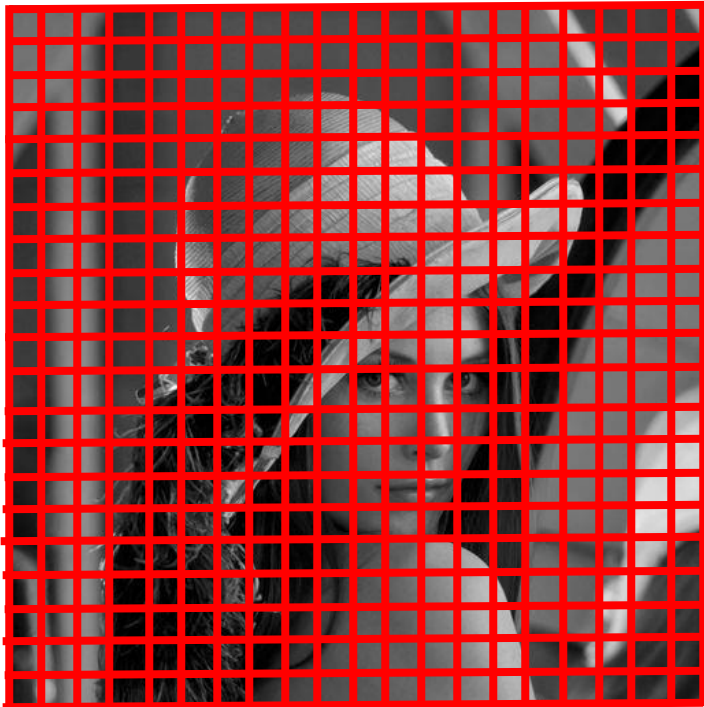


<https://classroom.udacity.com/courses/ud810/>

Trabajamos sobre imágenes digitales!

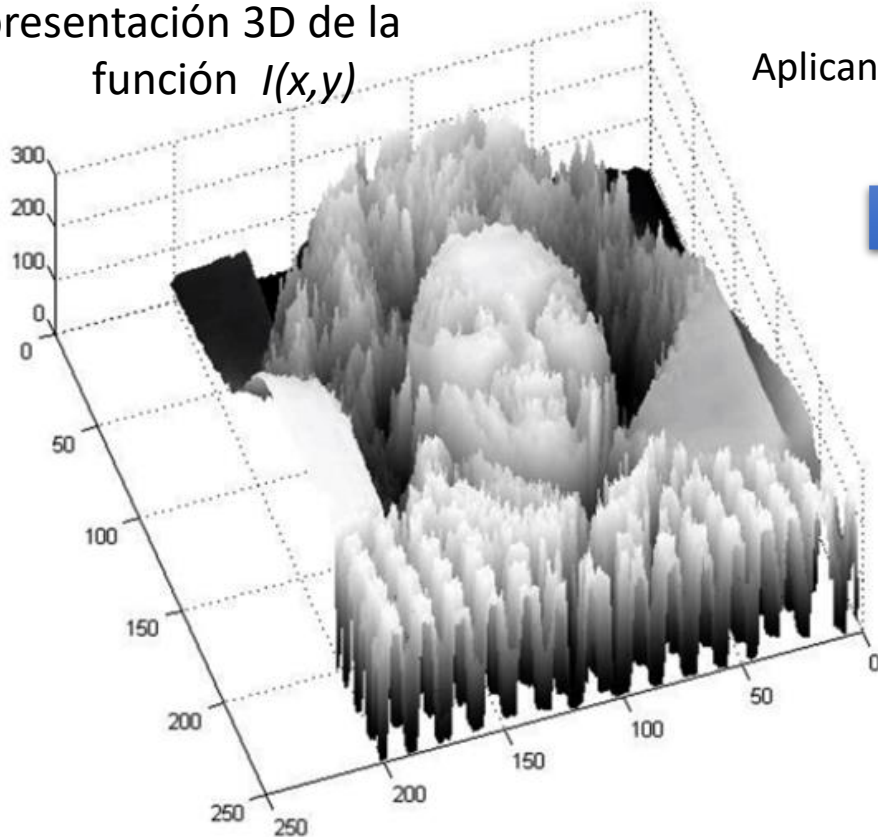


Trabajamos sobre imágenes digitales!

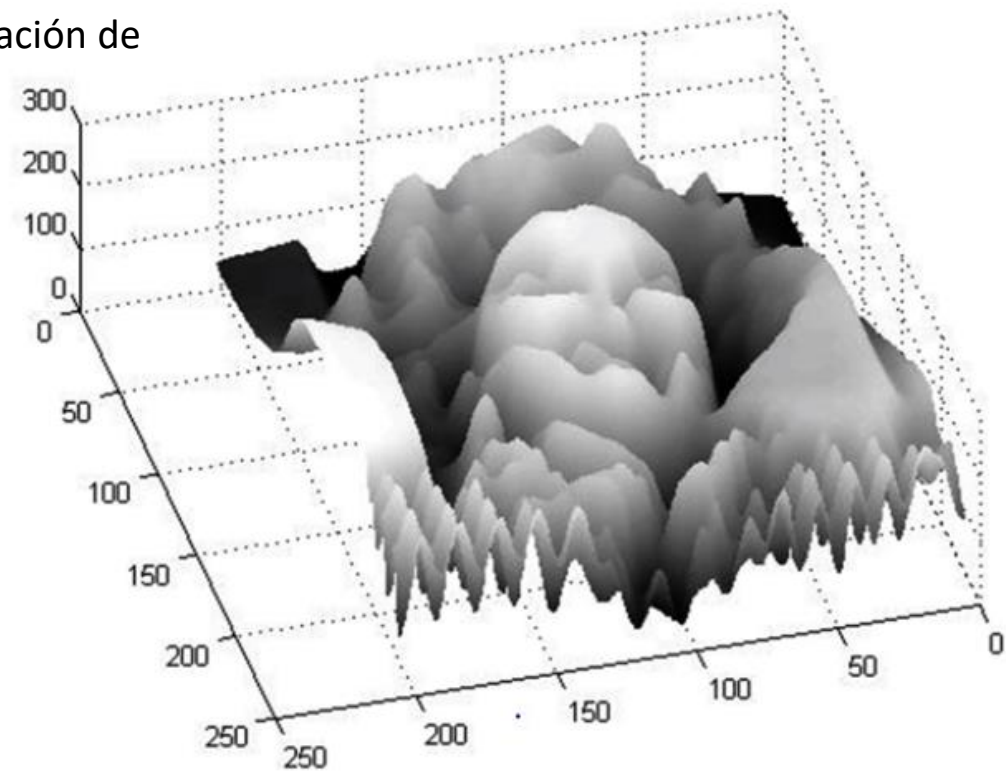


Se muestra en una malla 2D
Cada valor es ***cuantizado***!

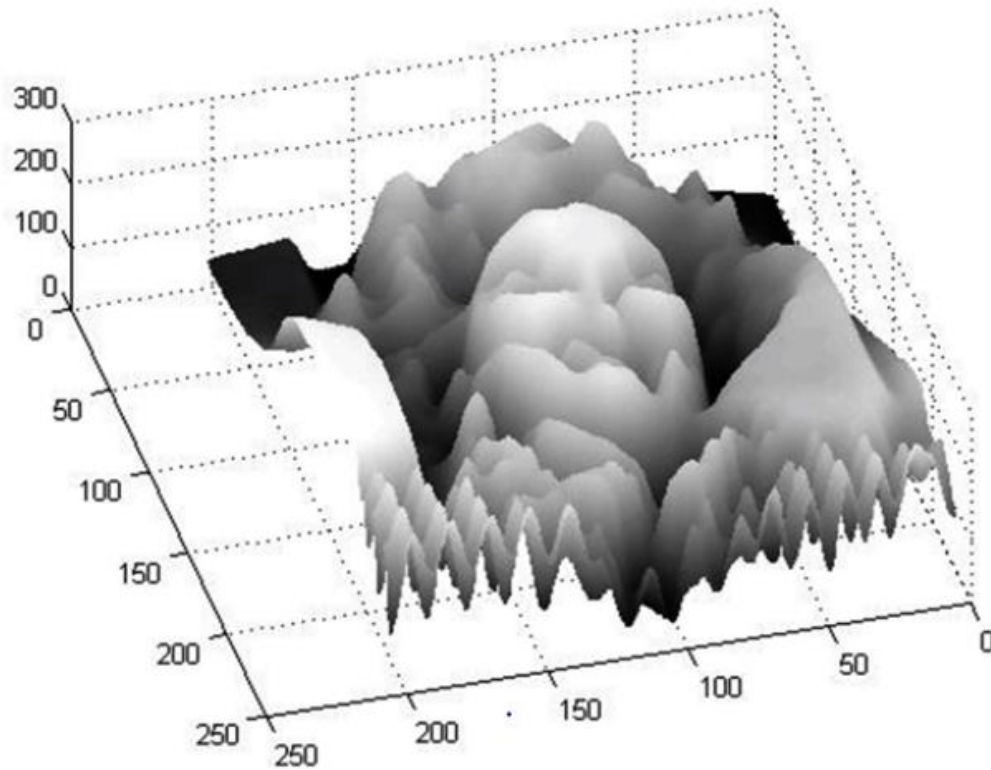
Representación 3D de la
función $I(x,y)$



Aplicando una operación de
suavizado



Si representamos de nuevo la
función como una imagen



Obtenemos una imagen
borrosa

<https://classroom.udacity.com/courses/ud810/>

Una imagen a color es una composición
de tres funciones

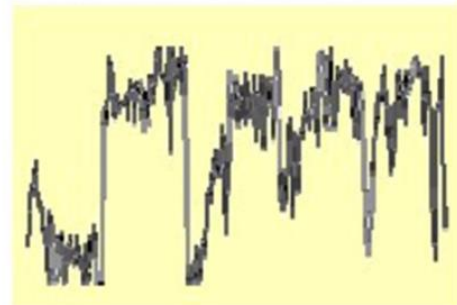
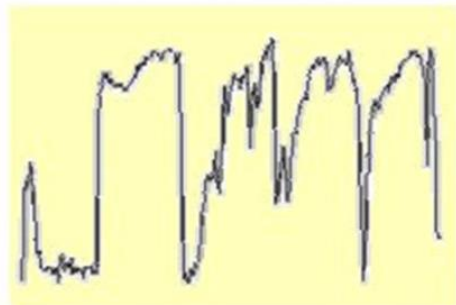
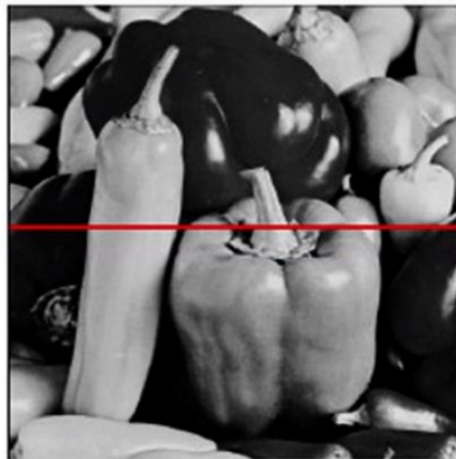


<http://www.augustrs.com/fractal-camouflage/>

$$f(x, y) = \begin{bmatrix} r(x, y) \\ g(x, y) \\ b(x, y) \end{bmatrix}$$

$$f: R \times R \rightarrow R^3$$

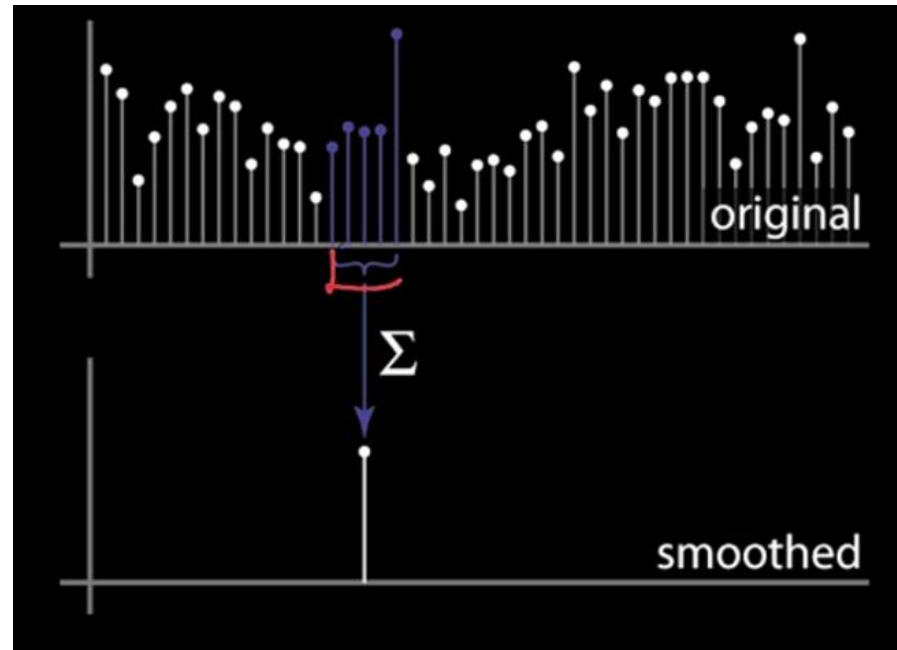
Ruido Gaussiano



$$f(x, y) = \underbrace{\hat{f}(x, y)}_{\text{Ideal Image}} + \underbrace{\eta(x, y)}_{\text{Noise process}}$$

Gaussian i.i.d. ("white") noise:
 $\eta(x, y) \sim \mathcal{N}(\mu, \sigma)$

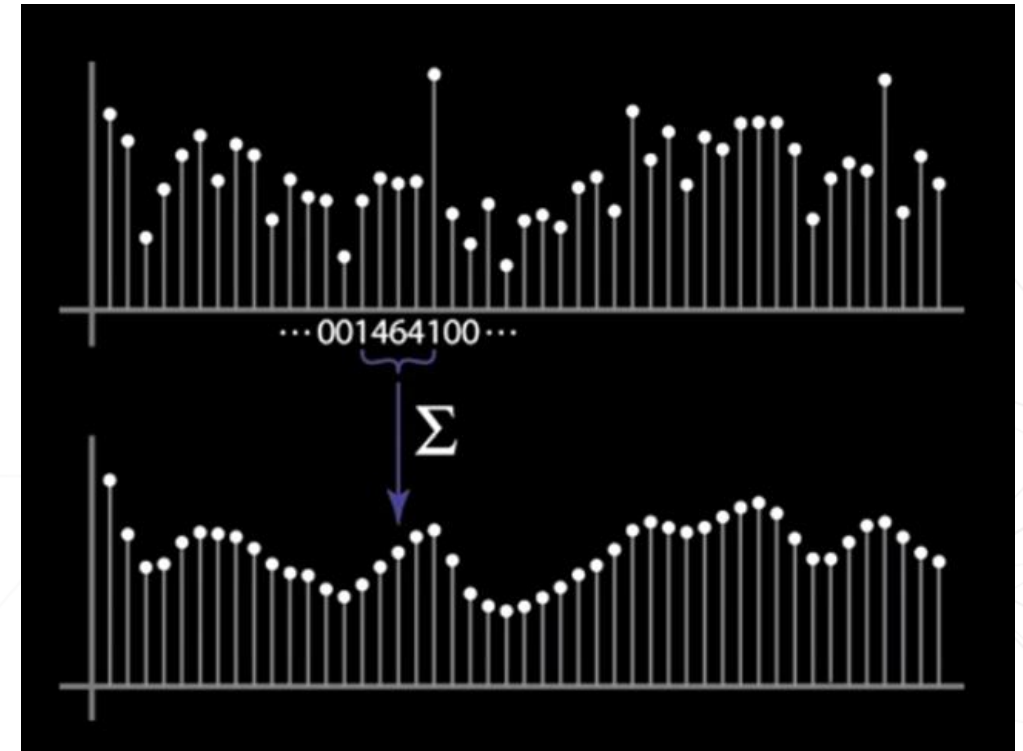
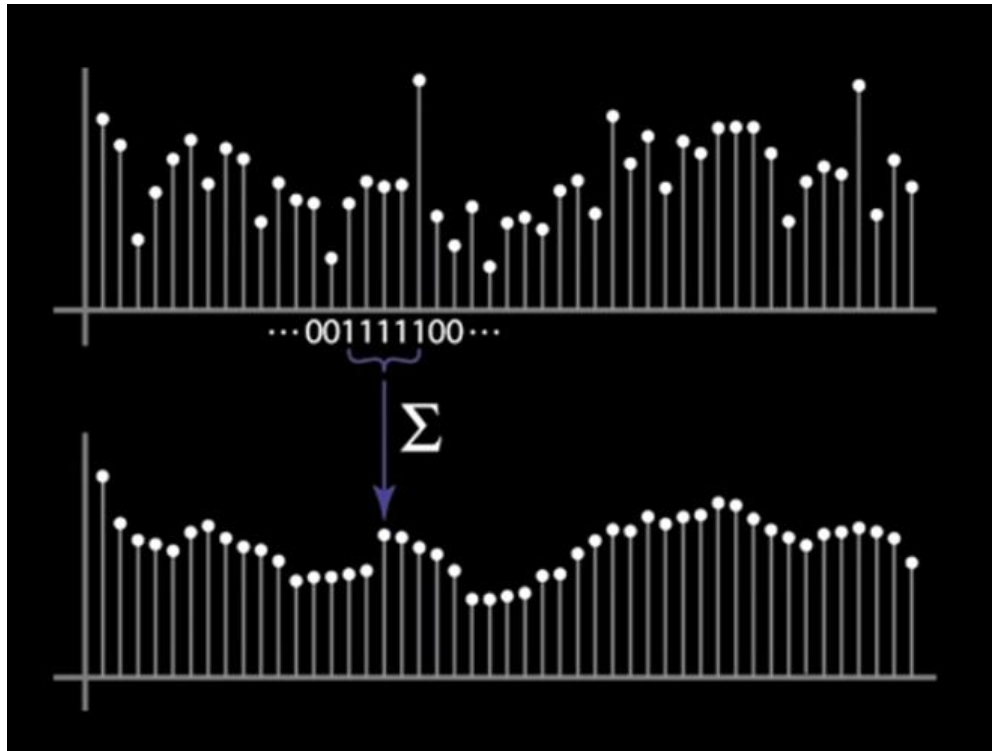
Qué tal si queremos reducir el ruido de una imagen?



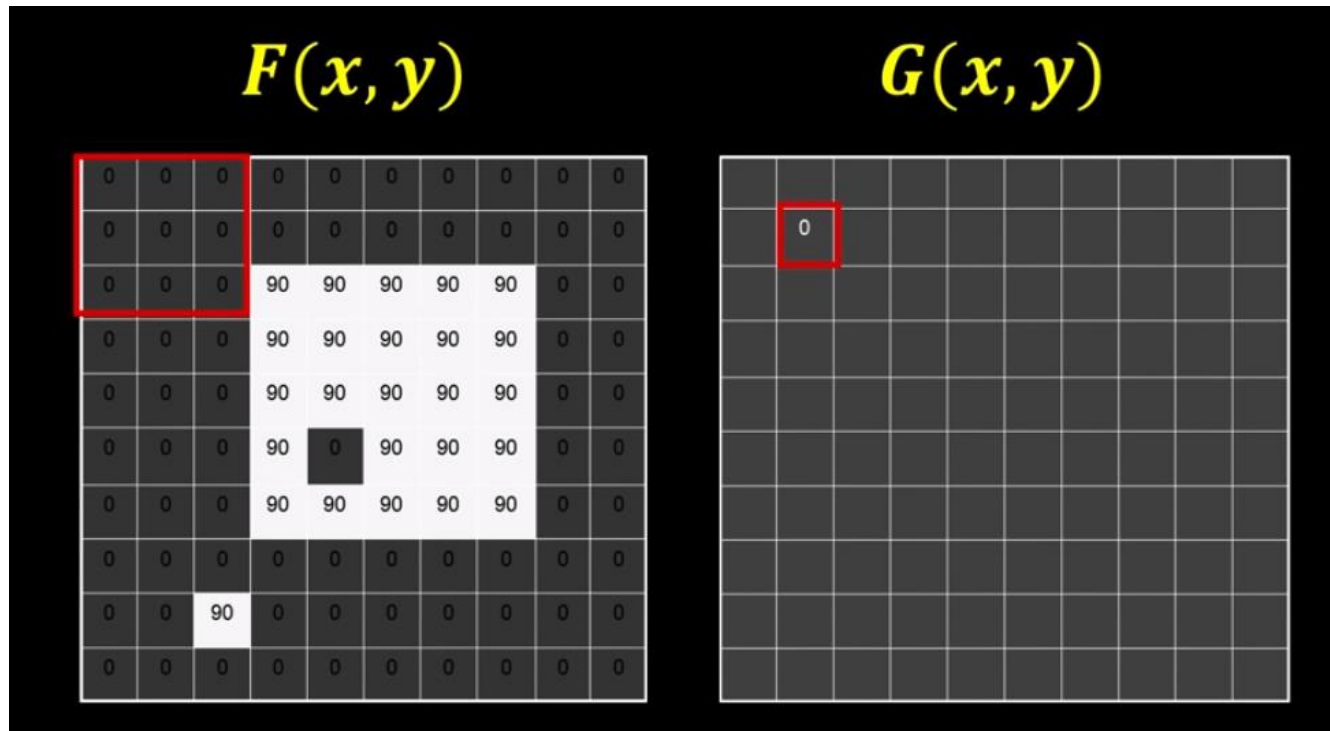
Asumimos:
*Los pixeles vecinos son **parecidos***
*El ruido es **aleatorio***

Media móvil!

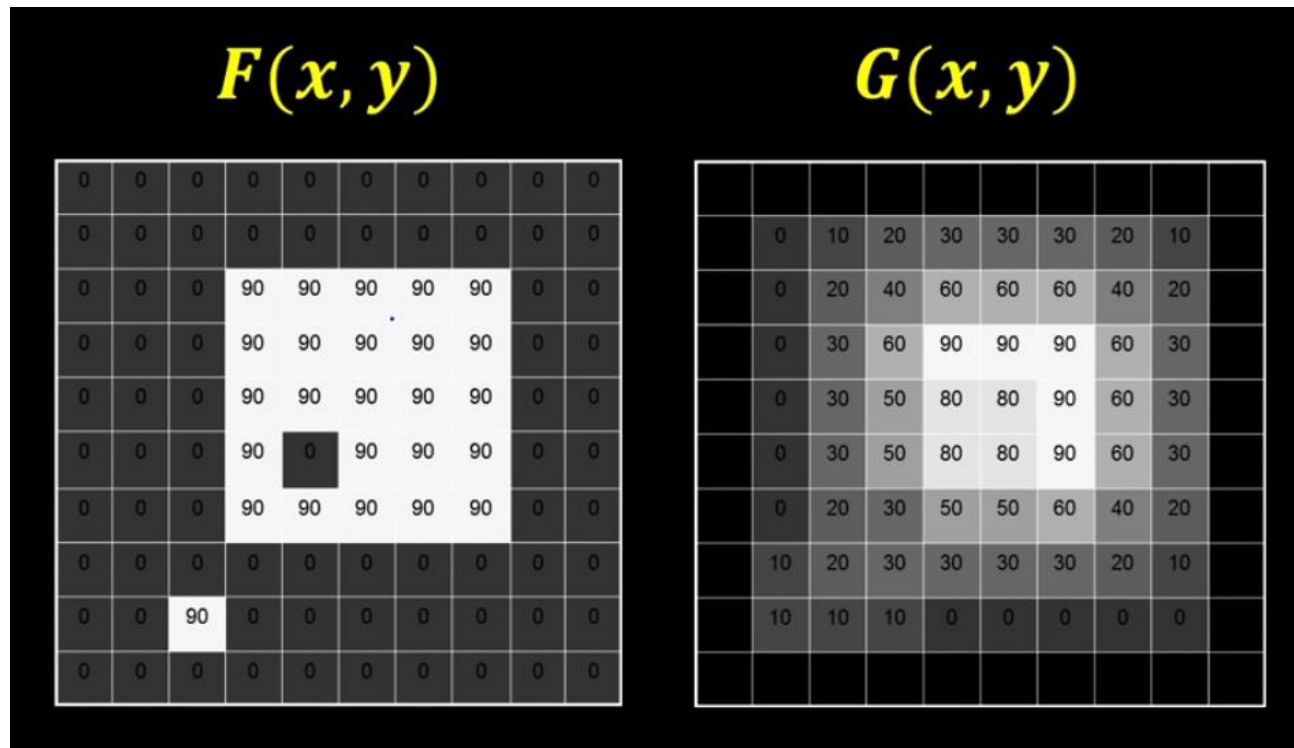
Media móvil ponderada



Media móvil ponderada en 2D



Media móvil ponderada en 2D



Filtrado (Correlación)

Realmente lo que hacemos es una correlación (filtrado)

Ponderación uniforme

$$G[i, j] = \frac{1}{(2k + 1)^2} \sum_{u=-k}^k \sum_{v=-k}^k F[i + u, j + v]$$

Suponemos que el ancho de la ventana es de la forma
 $2k+1$

Filtrado (Correlación)

Realmente lo que hacemos es una correlación (filtrado)

Ponderación no uniforme

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

Esto es una correlación cruzada!

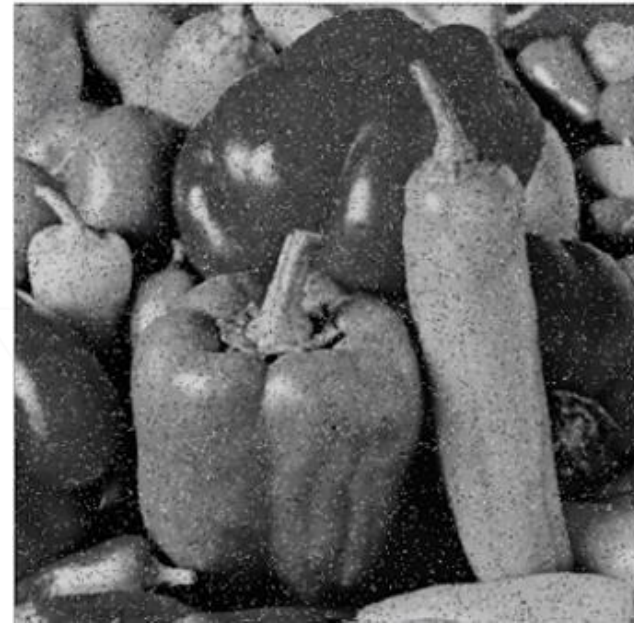
$$G = H \otimes F$$

H se conoce como kernel, máscara, etc.

Tipos de ruido



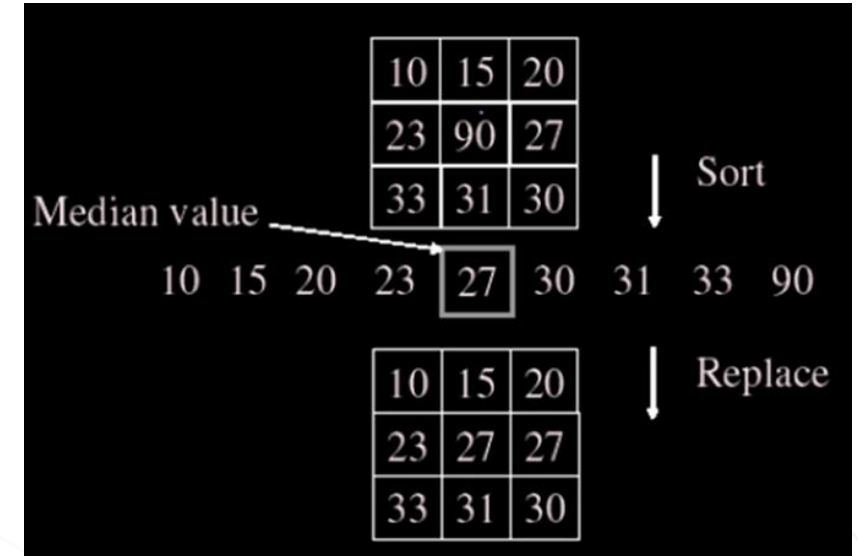
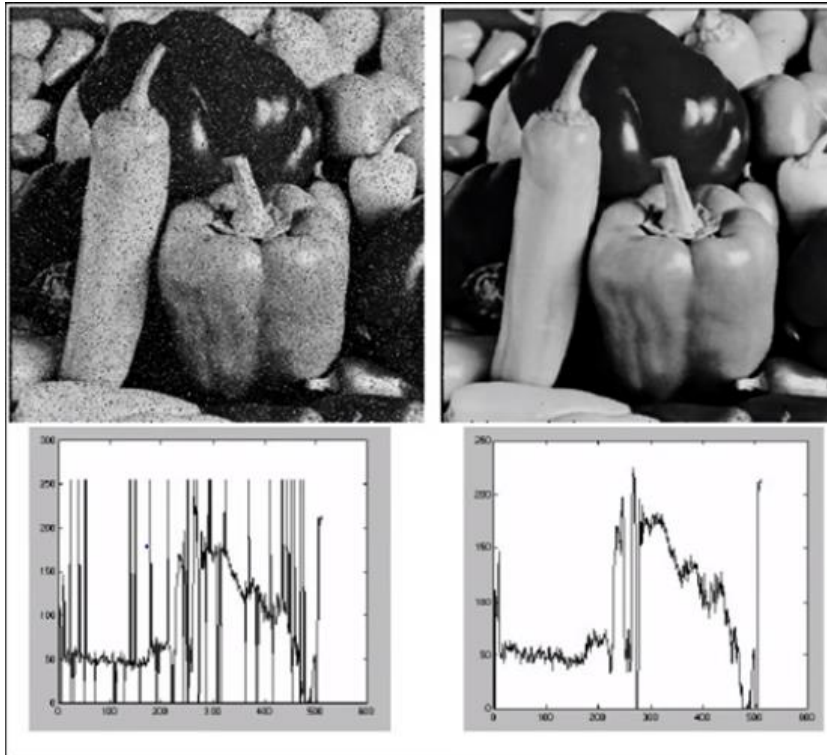
Ruido Gaussiano



Ruido sal y pimienta

Convolución y linealidad

Filtro ideal para este problema: Filtro mediana



Convolución y linealidad

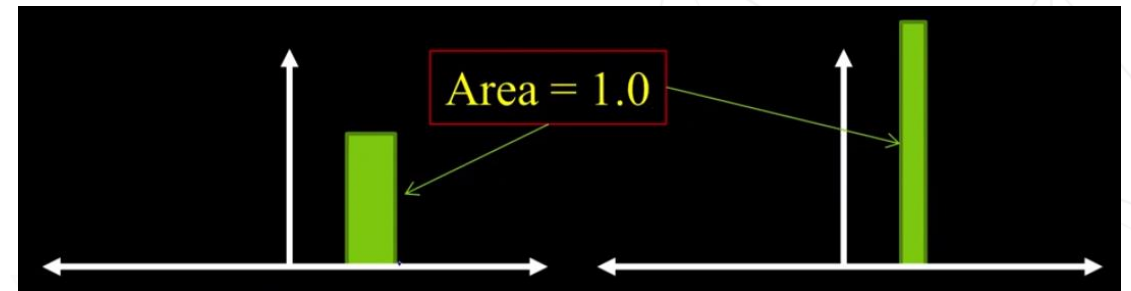
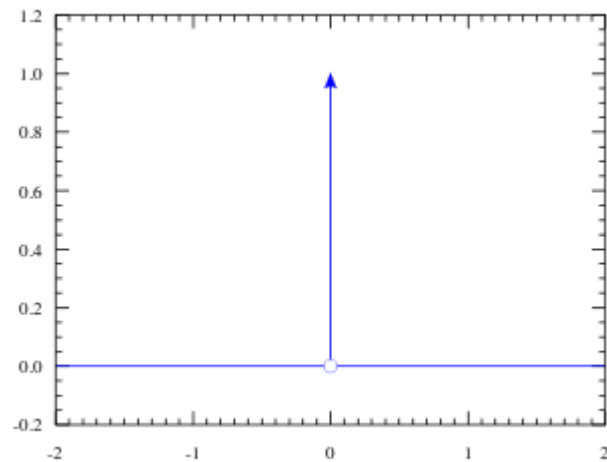
Un operador se denomina lineal si:

1. $H(f_1+f_2)=H(f_1)+H(f_2)$
2. $H(af_1)=aH(f_1)$

Esto implica que las operaciones de filtrado que hemos hecho hasta ahora son lineales!

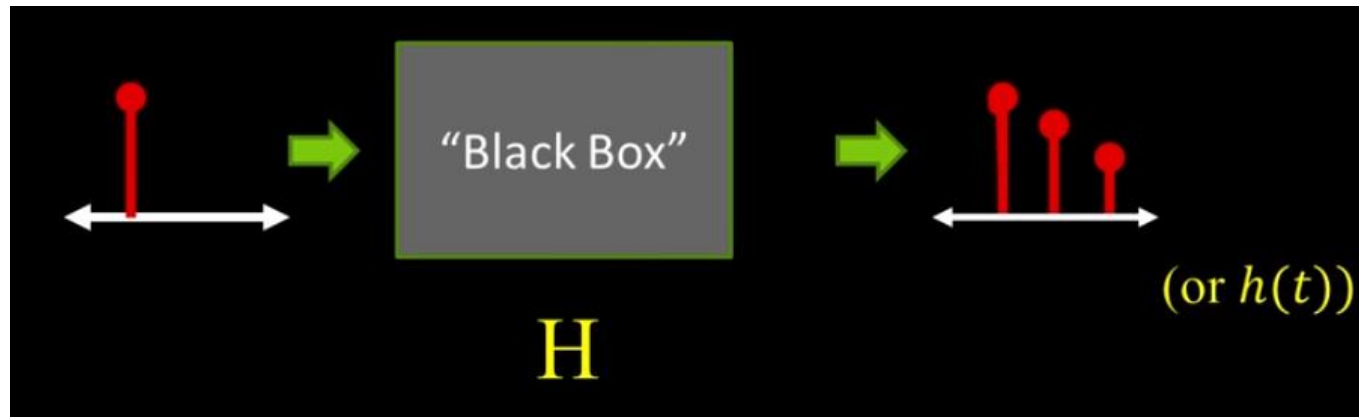
Convolución y linealidad

La función de impulso



Convolución y linealidad

La función de impulso: **Importancia**



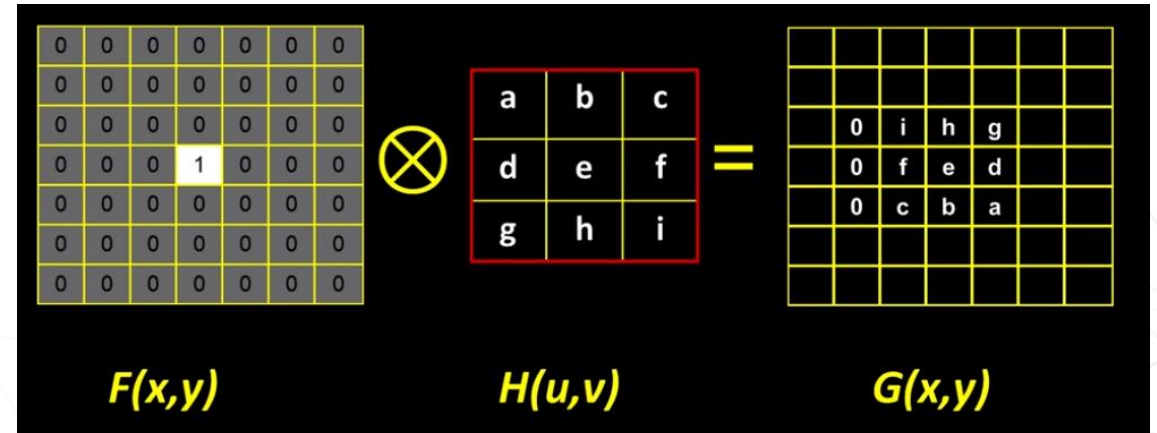
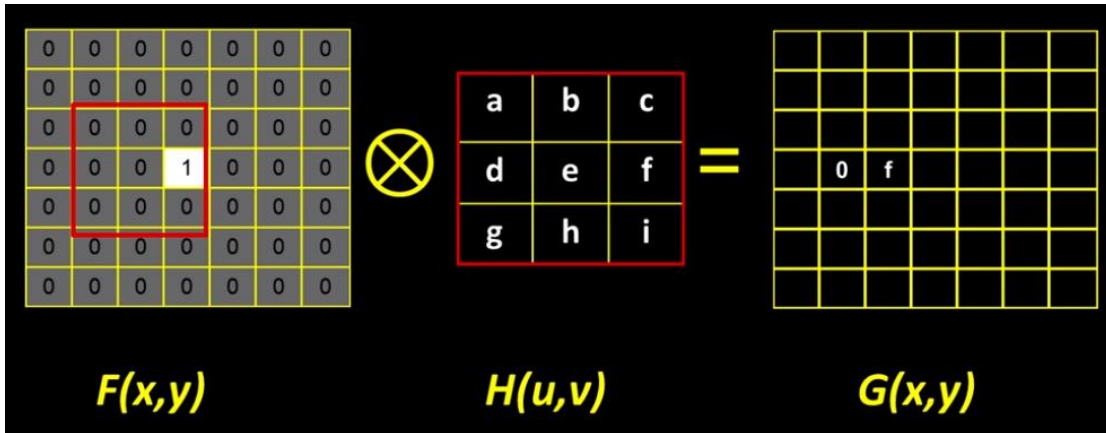
Función de impulso

Función de respuesta impulso

Conociendo esta función podremos predecir qué va a ocurrir con cualquier otra señal operada por H .

Convolución y linealidad

Filtrado de una respuesta al impulso



Cuántas operaciones debe llevar a cabo para calcular el filtro de una imagen $N \times N$ si empleo un kernel $M \times M$?

Convolución y linealidad

Correlación vs convolución

Cross-correlation

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i + u, j + v]$$

$$G = H \otimes F$$

Convolution

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

$$G = H \star F$$

Cuál es la diferencia entre una correlación y una convolución si el kernel es simétrico?

Centered at zero!

$$G[i, j] = \sum_{u=-k}^k \sum_{v=-k}^k H[u, v] F[i - u, j - v]$$

$$G = H * F$$

La convolución evita el *flip* que le hacemos al filtro, cuando operamos sobre una función de impulso.

Convolución y linealidad

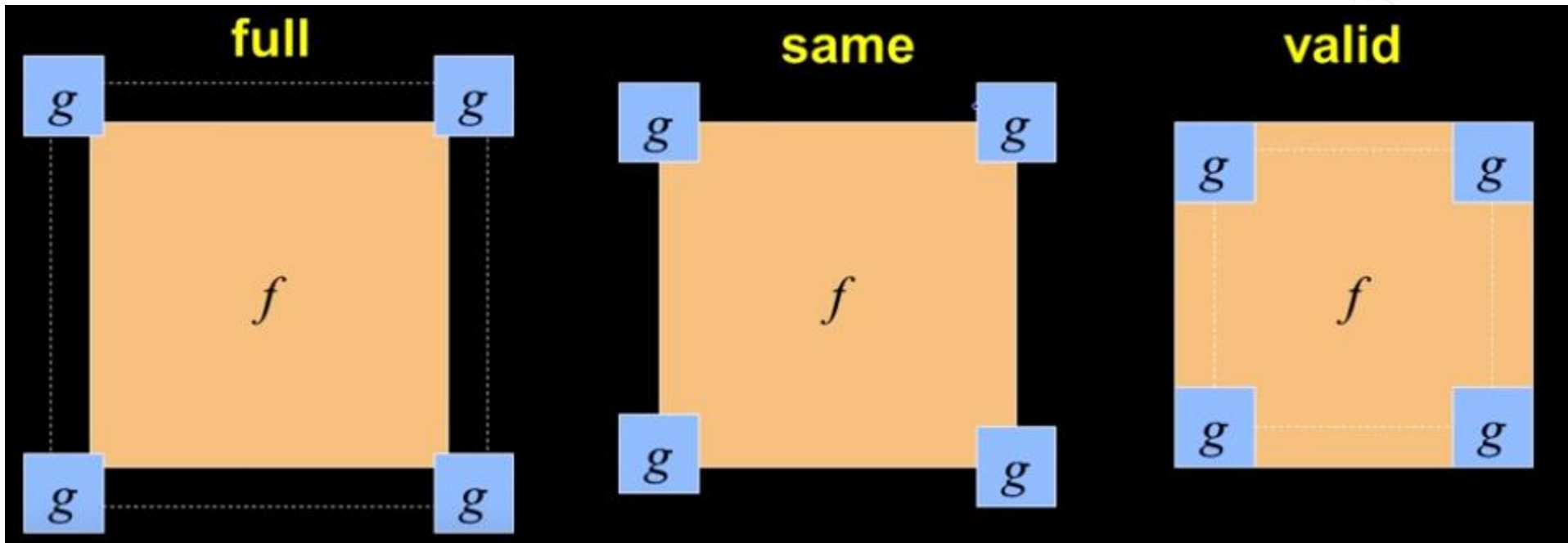
Propiedades de la convolución

1. Invariante al desplazamiento.
2. **Conmutativa.**
 $f * g = g * f$
3. **Asociativa.**
 $(h * f) * g = h * (g * f)$
4. **Identidad**
 $f * i = f$
5. **Diferenciación**

$$\frac{\partial}{\partial x}(f * g) = \frac{\partial f}{\partial x} * g$$

Convolución y linealidad

Cuándo aplicamos un filtro, qué pasa con el tamaño de la imagen resultante?



Convolución y linealidad

Métodos para “llenar el espacio necesario para obtener el mismo tamaño en la imagen de salida?”

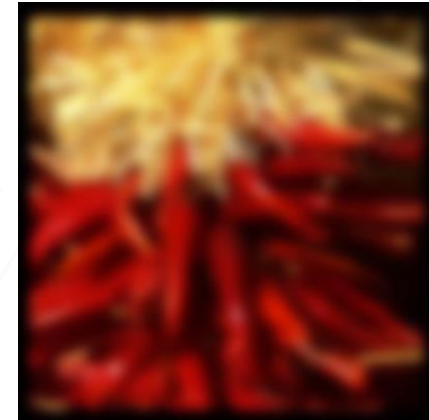
Envolver alrededor



Convolución y linealidad

Métodos para “llenar el espacio necesario para obtener el mismo tamaño en la imagen de salida?”

Filtro clip



Convolución y linealidad

Métodos para “llenar el espacio necesario para obtener el mismo tamaño en la imagen de salida?”

Copiar el borde (replica)



Convolución y linealidad

Métodos para “llenar el espacio necesario para obtener el mismo tamaño en la imagen de salida?”

Reflejar a lo largo del borde (simétrico)



Filtro de nitidez:

Convolución y linealidad

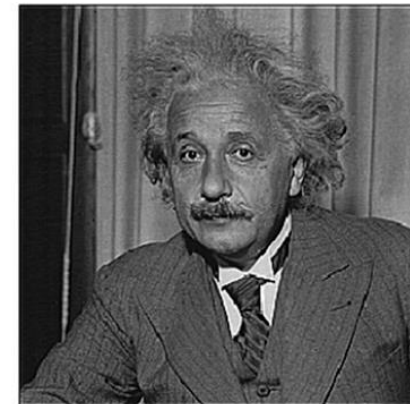
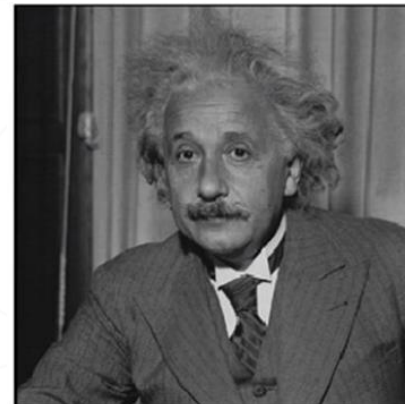
Filtro de nitidez



$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 0 \end{bmatrix} - \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix}$$

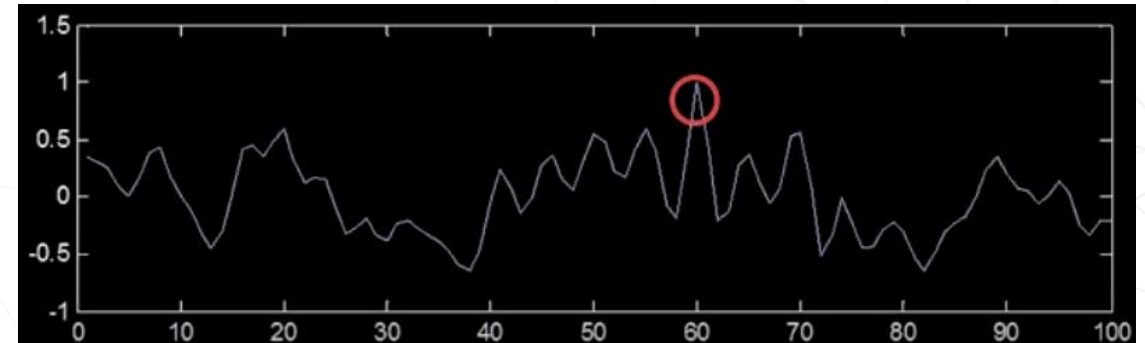
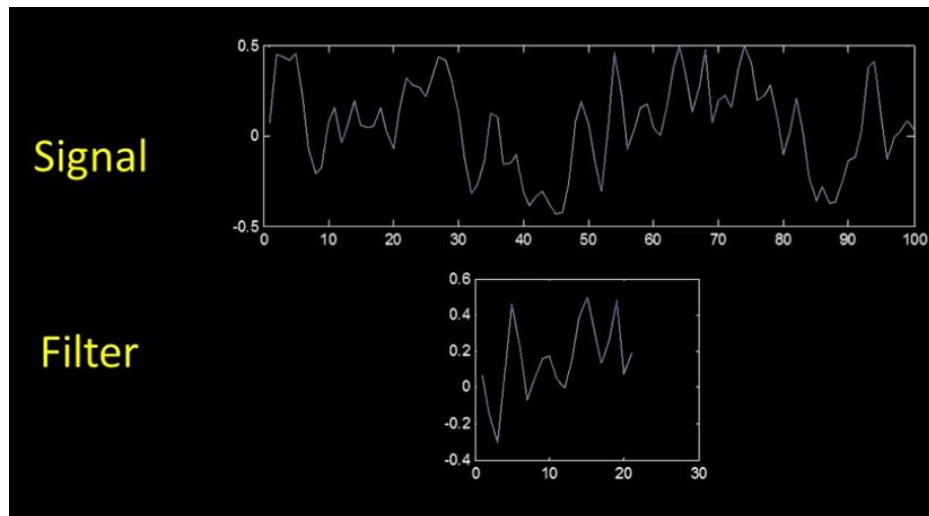


Unsharp mask?



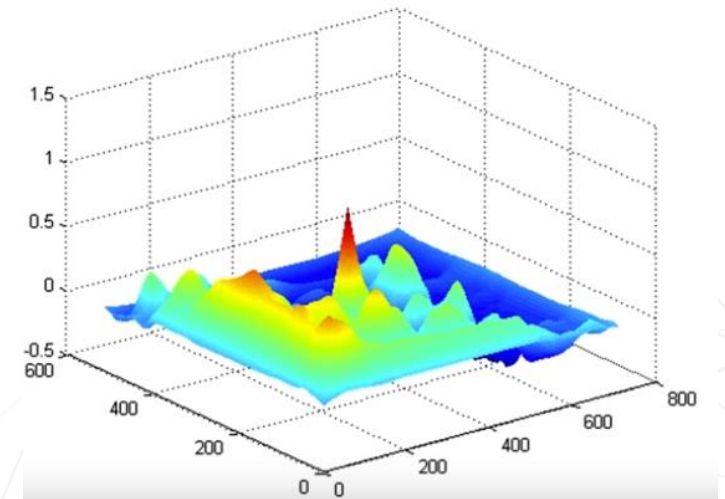
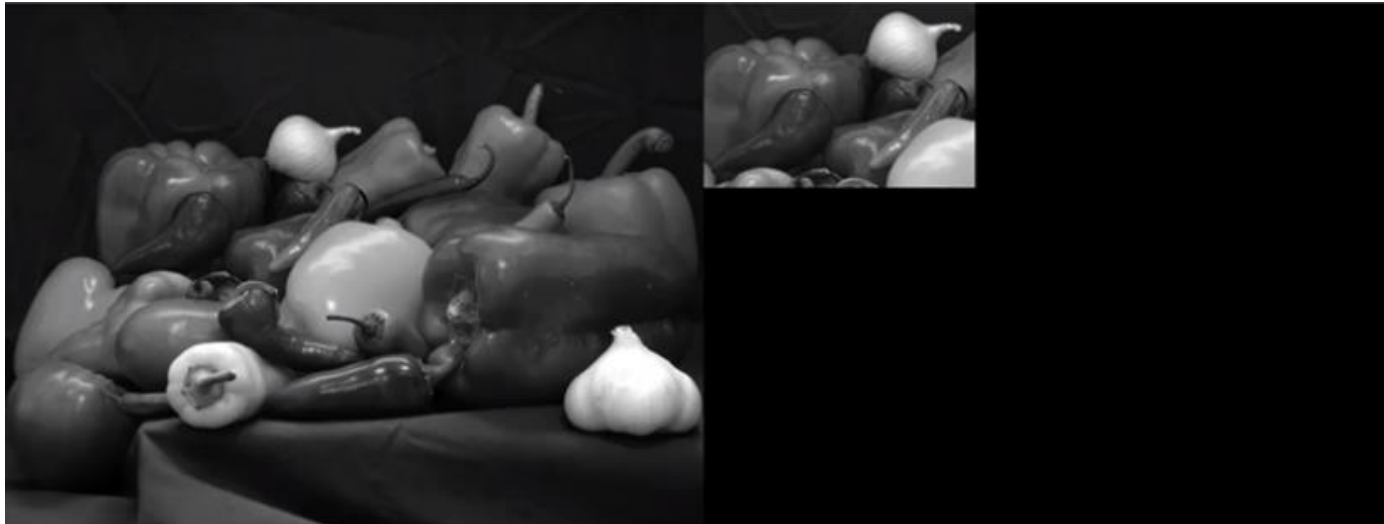
Filtros como plantillas

Correlación 1D



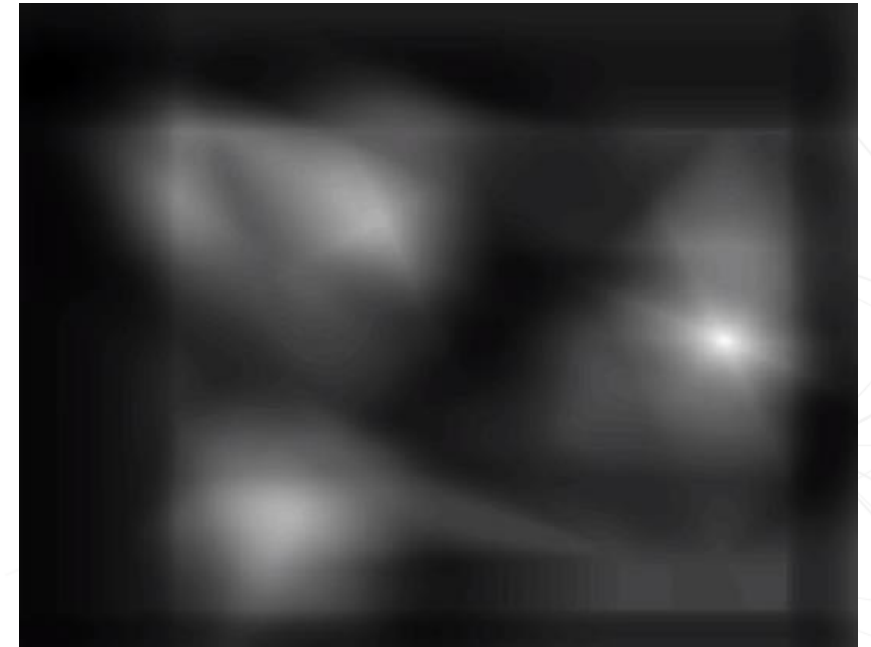
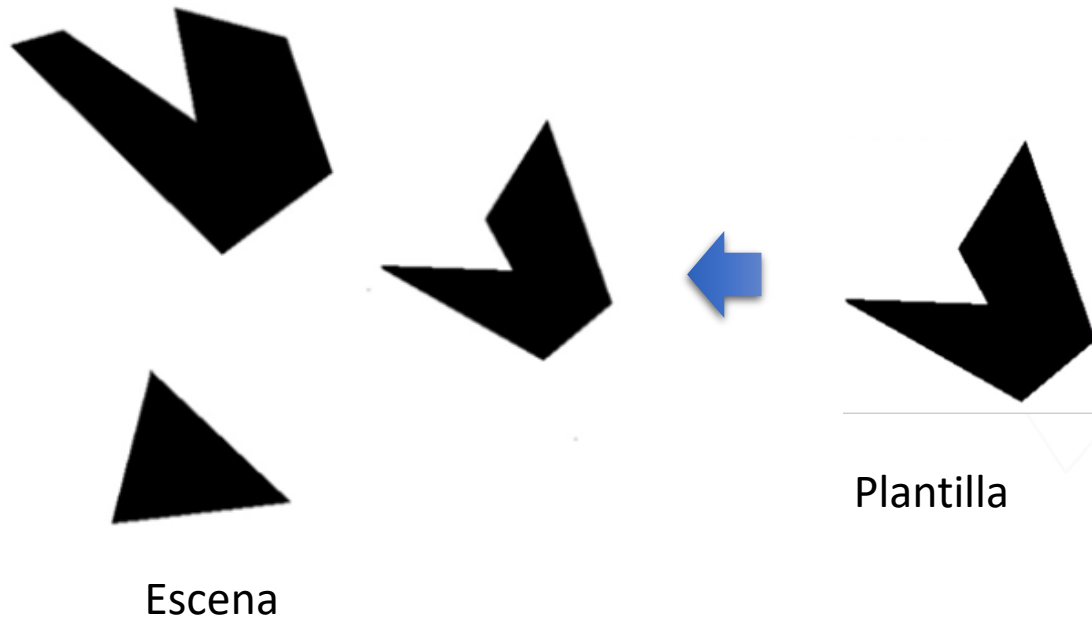
Filtros como plantillas

Correlación 2D



Filtros como plantillas

Correlación 2D (Ejemplo)



Mapa de correlación

Filtros como plantillas

No son buenas para todo!



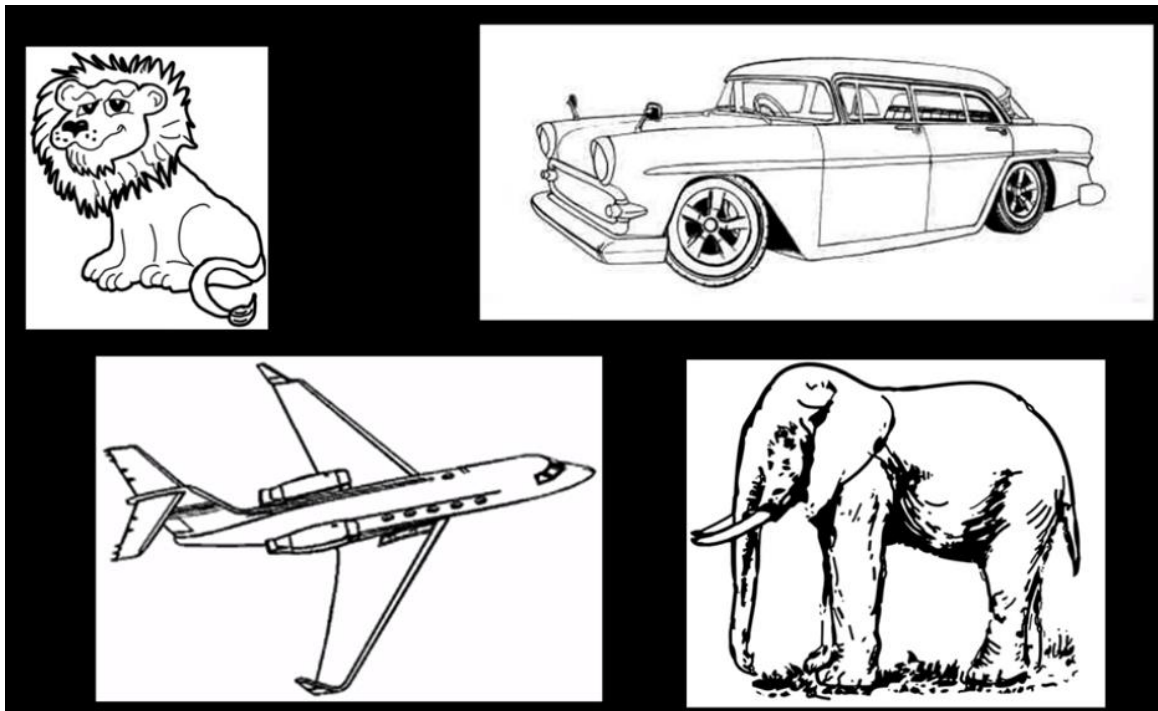
Filtros como plantillas

No son buenas para todo!



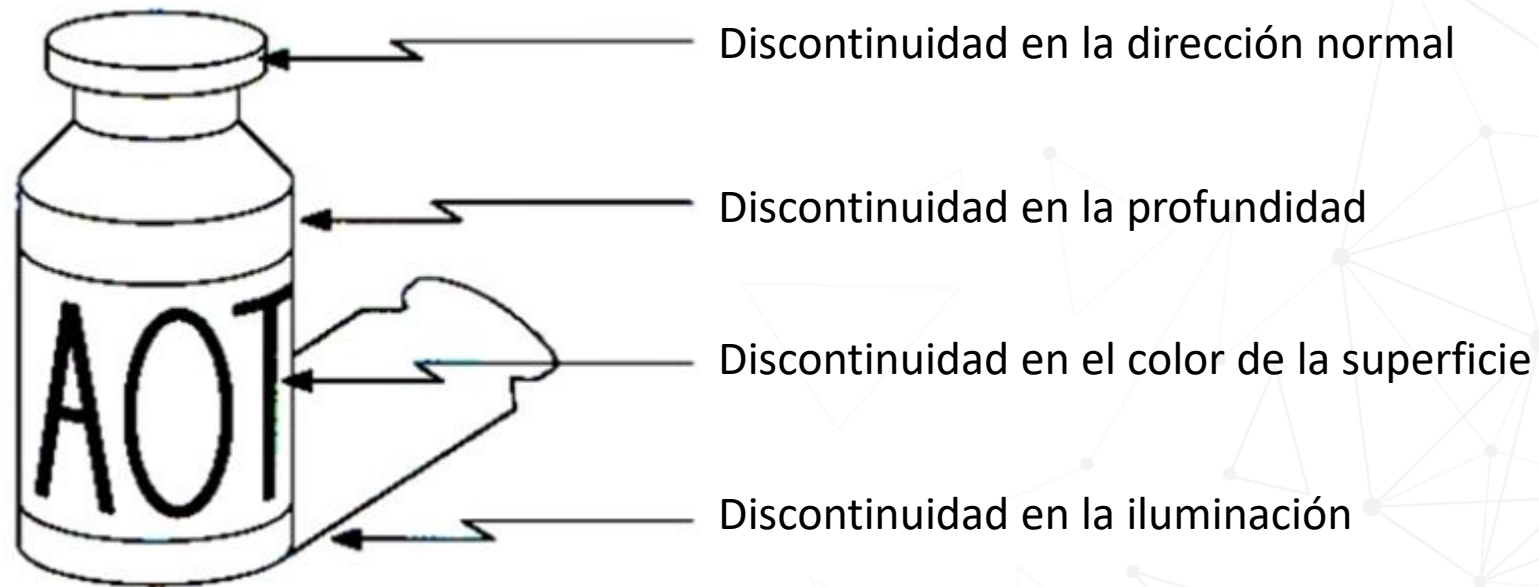
Detección de bordes

Los bordes nos dan información que puede ser muy útil



Detección de bordes

Los bordes nos dan información que puede ser muy útil



Detección de bordes

Los bordes nos dan información que puede ser muy útil



Discontinuidad en la dirección normal

Discontinuidad en la profundidad

Discontinuidad en el color de la superficie

Discontinuidad en la iluminación

Detección de bordes

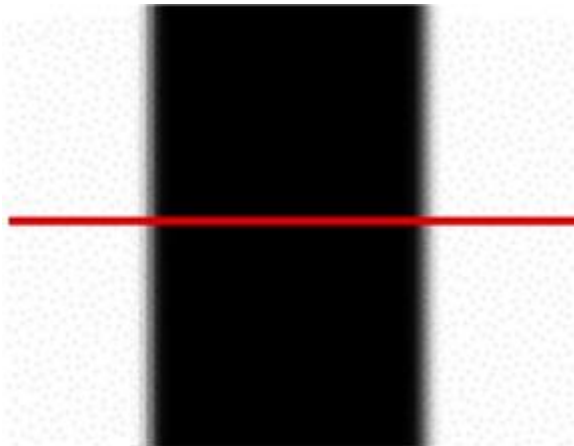
El problema de la detección de bordes consiste esencialmente en detectar grandes cambios en la imagen

81	82	26	24
82	33	25	25
81	82	26	24

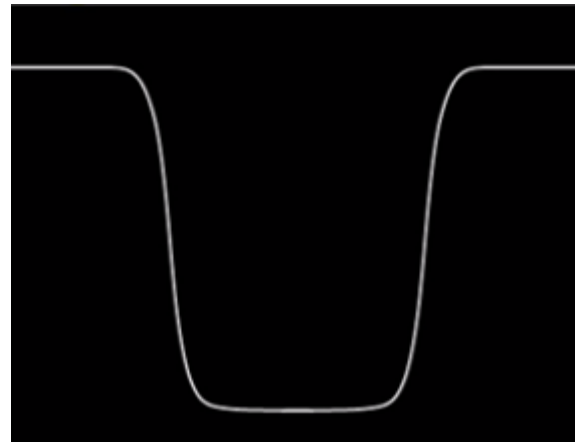
Qué tan grande debe ser el
vecindario?
Cómo detecto el cambio?

Detección de bordes

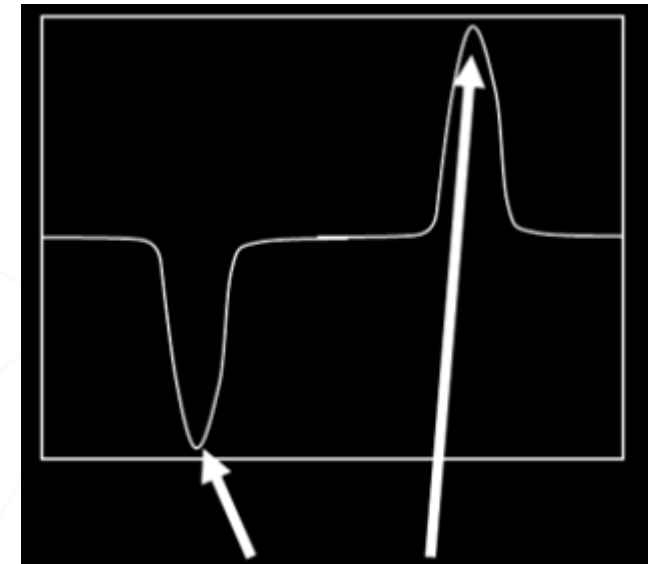
Derivadas y los bordes



Imagen



Perfil



*Los bordes se encuentran en los
puntos extremos de las derivadas!*

Detección de bordes

Operadores diferenciales

Cuando se aplican a imágenes, devuelven derivadas.

Se modelan como los kernels que permiten calcular la función de gradiente de la imagen.

Se umbraliza la función resultante para detectar los píxeles asociados a bordes

Ángulo del gradiente

$$\theta = \tan^{-1}\left(\frac{\partial f}{\partial y} / \frac{\partial f}{\partial x}\right)$$

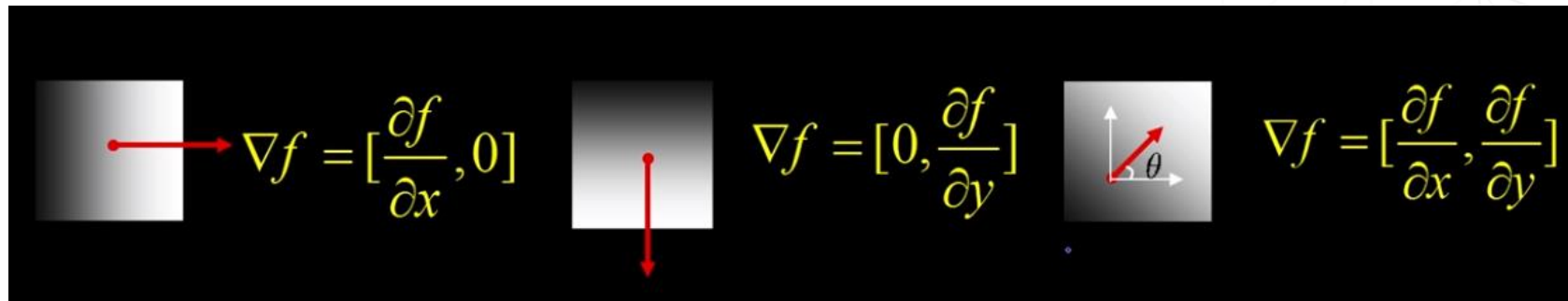
Detección de bordes

Gradiente de una imagen

$$\nabla f = \left[\frac{\partial f}{\partial x}, \frac{\partial f}{\partial y} \right]$$

Magnitud del gradiente

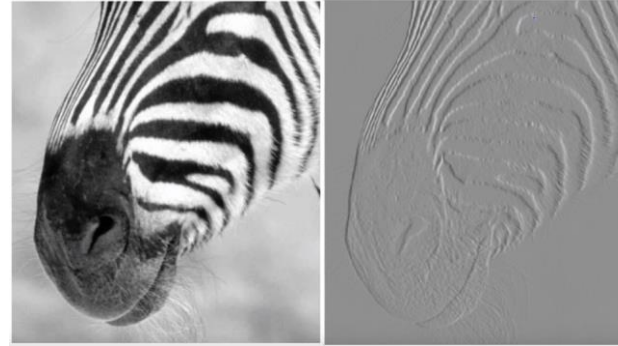
$$\|\nabla f\| = \sqrt{\left(\frac{\partial f}{\partial x}\right)^2 + \left(\frac{\partial f}{\partial y}\right)^2}$$



El gradiente apunta a la dirección del cambio más “rápido”.

Detección de bordes

Gradiente de una imagen



Horizontal o vertical?



$$\frac{\partial f(x, y)}{\partial x} = \lim_{\varepsilon \rightarrow 0} \frac{f(x + \varepsilon, y) - f(x, y)}{\varepsilon}$$

Realmente lo que podemos hacer en un computador es calcular este valor vía diferencias finitas:

$$\frac{\partial f(x, y)}{\partial x} \approx \frac{f(x + 1, y) - f(x, y)}{1}$$

Diferencias finitas a la derecha

https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_gradients/py_gradients.html

Detección de bordes

Gradiente de una imagen (qué **kernel** emplear?)

0	0	0
-1/2	0	+1/2
0	0	0

Este **kernel** nos permite calcular un promedio de una derivada a la derecha y una derivada a la izquierda vía correlación.

https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_gradients/py_gradients.html

Detección de bordes

Gradiente de una imagen: Operador ***Sobel***.

$$\frac{1}{8} * \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix} \\ S_x$$

$$\frac{1}{8} * \begin{bmatrix} 1 & 2 & 1 \\ 0 & 0 & 0 \\ -1 & -2 & -1 \end{bmatrix} \\ S_y$$

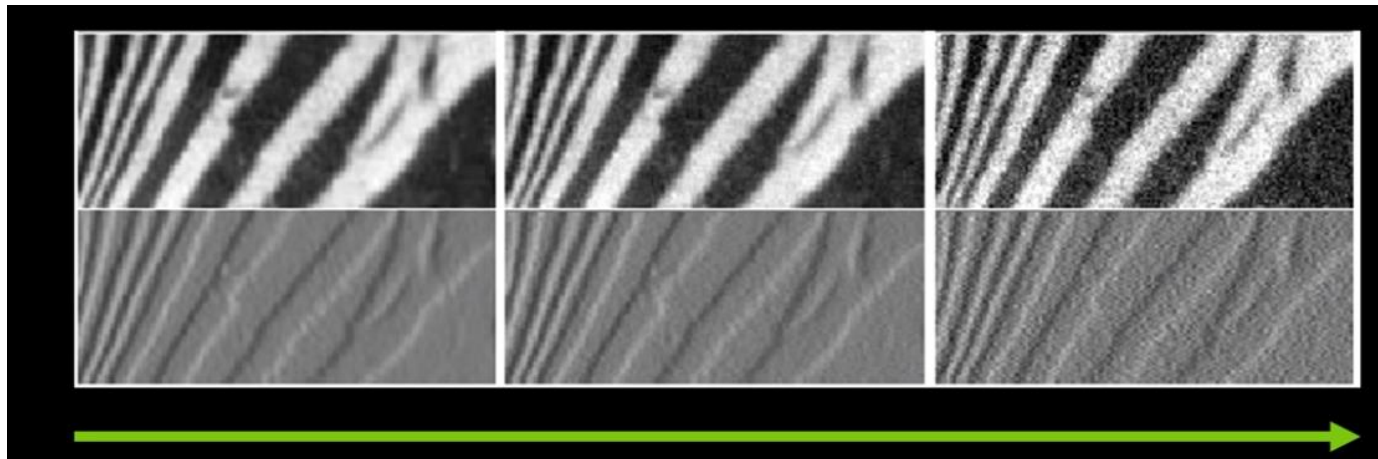


Este operador considera cada una de las derivadas, y los ***vecinos*** que aportan en su cálculo.

https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_gradients/py_gradients.html

Detección de bordes

Gradiente de una imagen: Operador ***Sobel***.



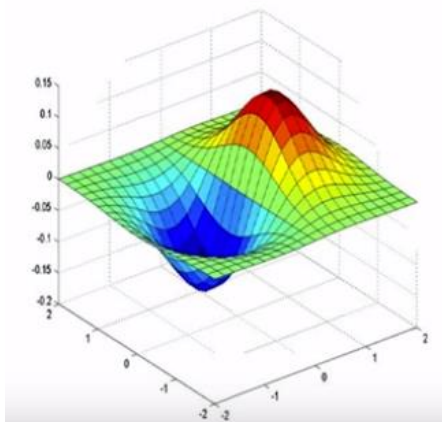
Qué pasa con el cálculo del gradiente si aumentamos el ruido?

Qué podemos hacer? Taller

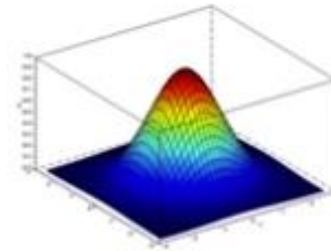
Detección de bordes

Gradiente de una imagen 2D

$$(I \otimes g) \otimes h_x = I \otimes (g \otimes h_x)$$



Derivada del filtro Gaussiano



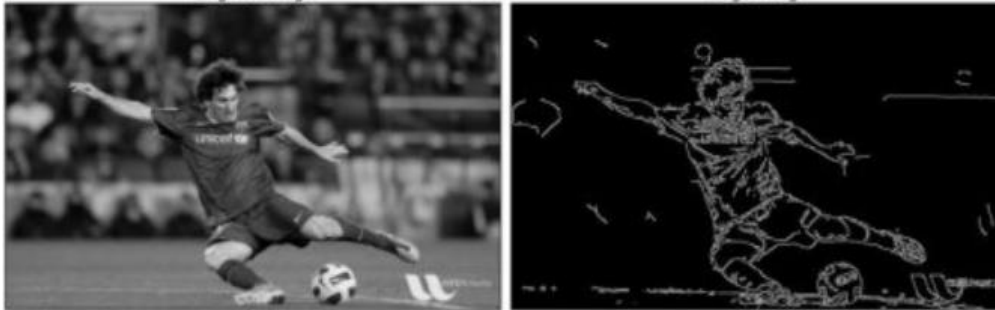
0.0030	0.0133	0.0219	0.0133	0.0030
0.0133	0.0596	0.0983	0.0596	0.0133
0.0219	0.0983	0.1621	0.0983	0.0219
0.0133	0.0596	0.0983	0.0596	0.0133
0.0030	0.0133	0.0219	0.0133	0.0030

Kernel Gaussiano

**Efectos del σ en el cálculo del filtro.
Taller.**

Detección de bordes

Gradiente de una imagen 2D: Detección de bordes de Canny.



Es un algoritmo de múltiples etapas, que considera la reducción de ruido, el cálculo de la intensidad del gradiente de la imagen, supresión de puntos que no corresponden a bordes y una clasificación de bordes basada en ***histéresis***.

https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_canny/py_canny.html

Transformada de Hough

Modelos paramétricos

Estos modelos pueden representar una clase de muestras donde cada clase está definida por un valor de los parámetros.

Ejemplos: Líneas, círculos o plantillas.

La clave del modelo a identificar en la imagen es representarlo de una forma matemática!

Transformada de Hough

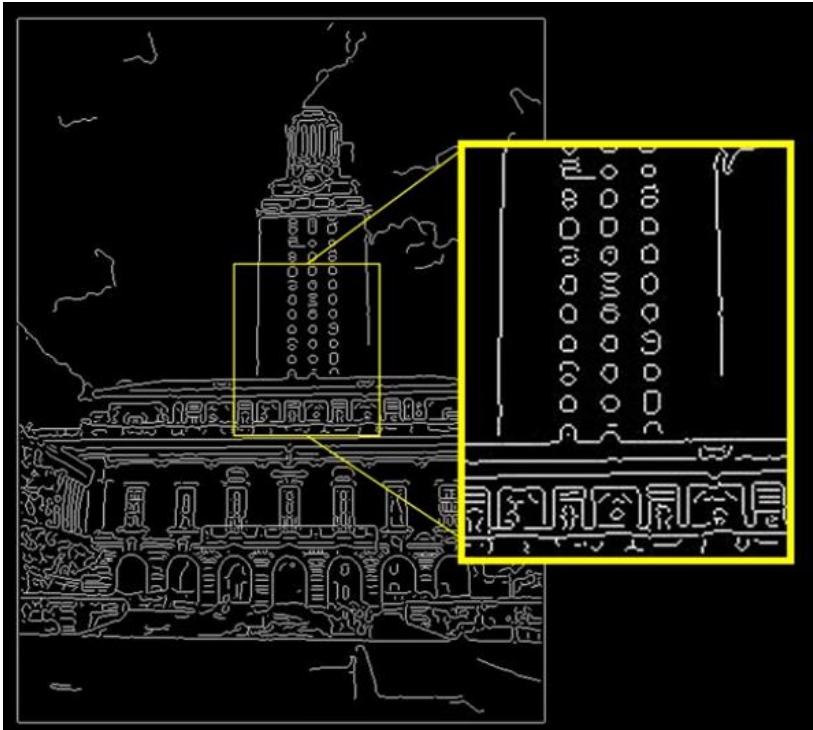
Ajuste de líneas



El objetivo es encontrar estas líneas (rectas) en las imágenes

Transformada de Hough

Ajuste de líneas



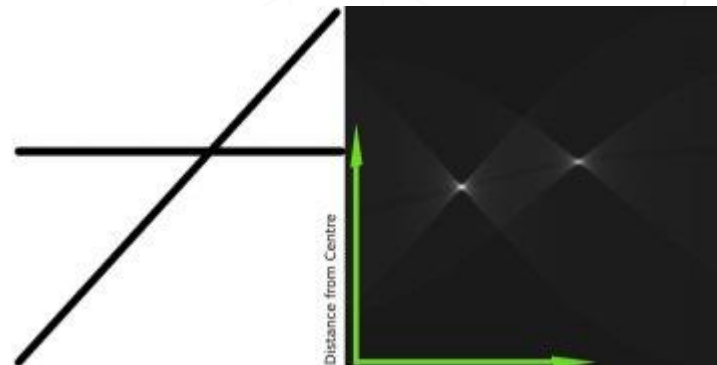
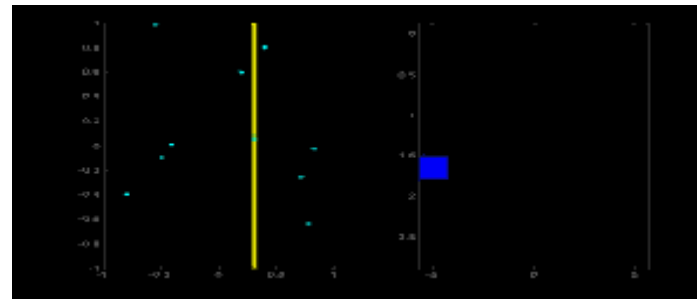
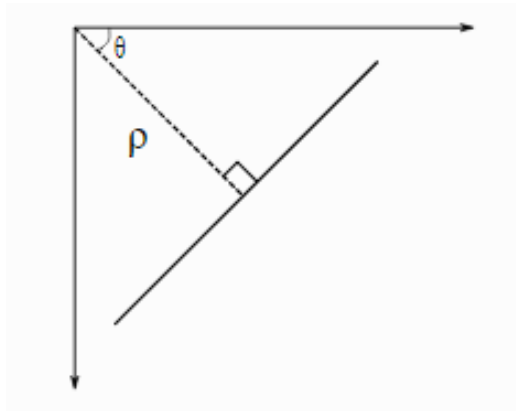
El problema es difícil si consideramos que hay puntos que no tienen que ver con las imágenes.

Algunas líneas no están completas.

El ruido también es detectado después de una operación de detección de bordes.

Transformada de Hough

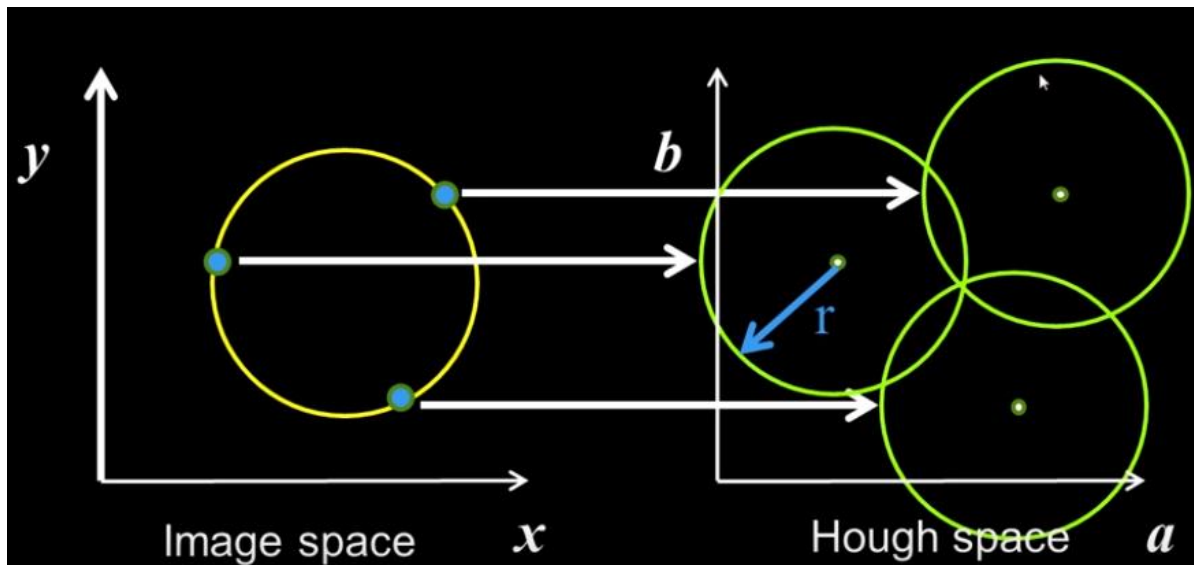
Ajuste de líneas



https://opencv-python-tutroals.readthedocs.io/en/latest/py_tutorials/py_imgproc/py_houghlines/py_houghlines.html

Transformada de Hough

Ajuste de círculos

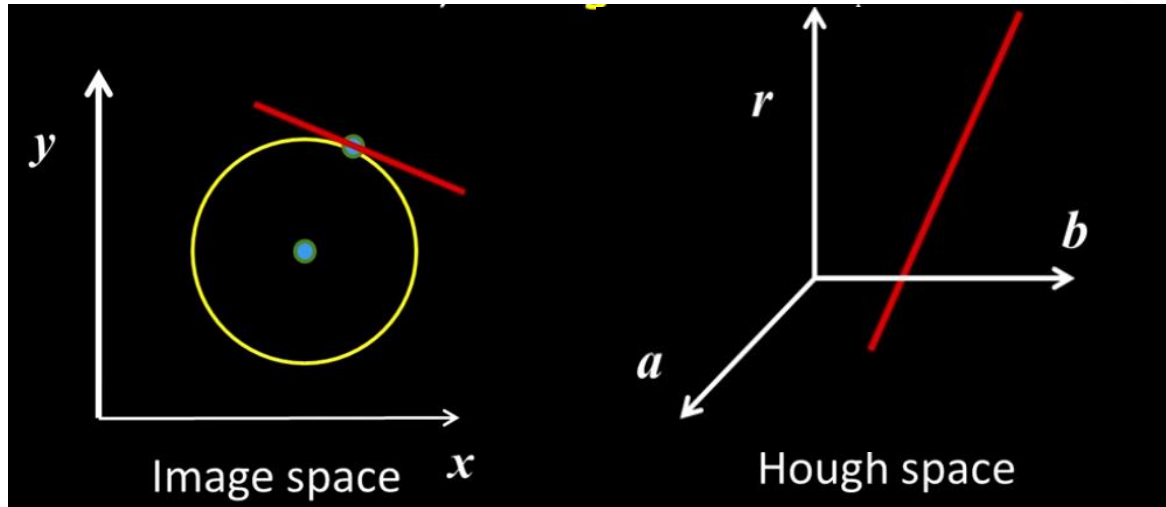
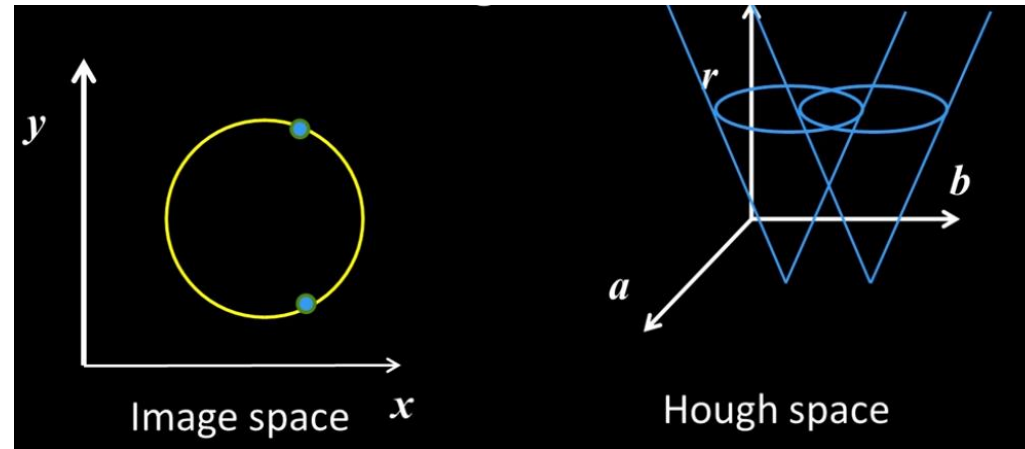


La mayoría de los votos los tendremos
en el centro del círculo!

$$(x - a)^2 + (y - b)^2 = r^2$$

Transformada de Hough

Ajuste de círculos



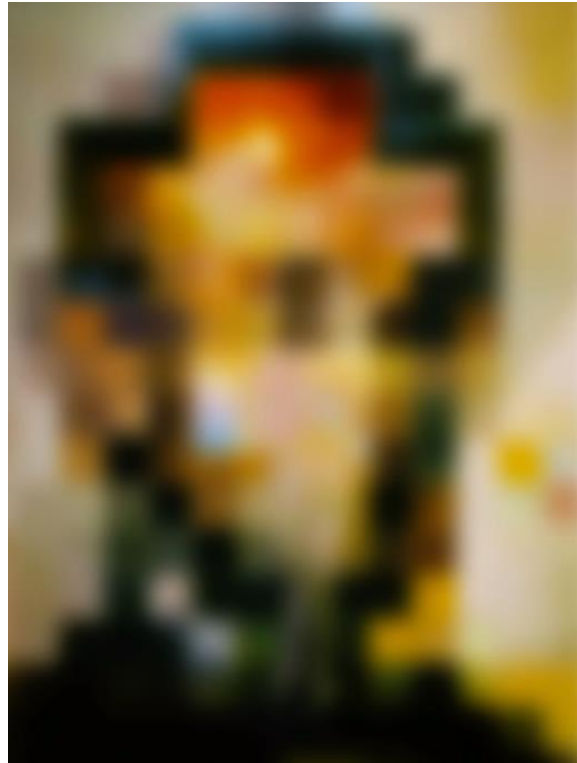
Como la votación es en 3D, el proceso puede ser muy complejo computacionalmente!

Si aprovechamos el gradiente, la votación se puede hacer más fácilmente!

$$(x - a)^2 + (y - b)^2 = r^2$$

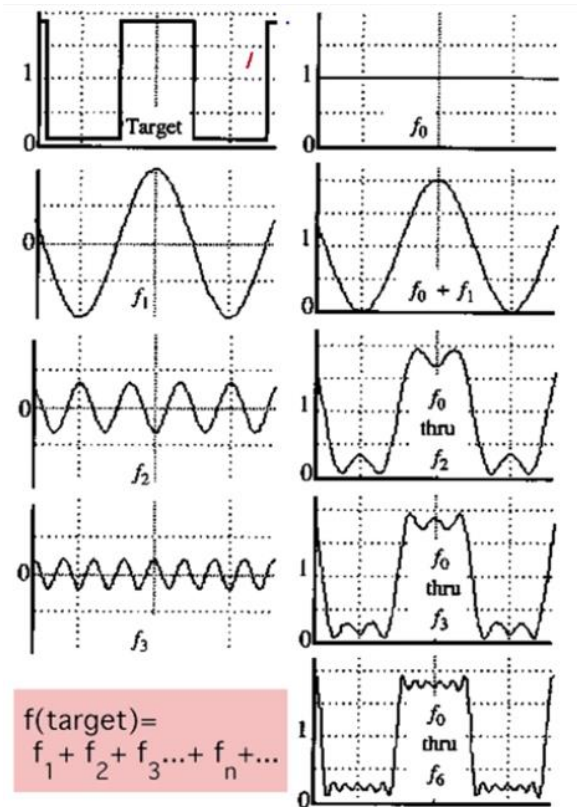
Transformada de Fourier

Nos permite representar una función en un conjunto de funciones base.
Es decir, tenemos la información de frecuencias espaciales.



Y podemos llevar a cabo
filtrado en frecuencias.

Transformada de Fourier



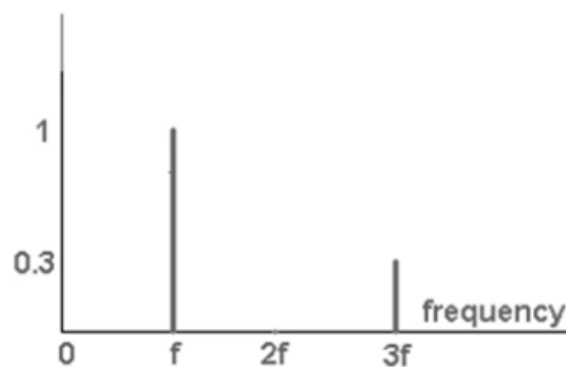
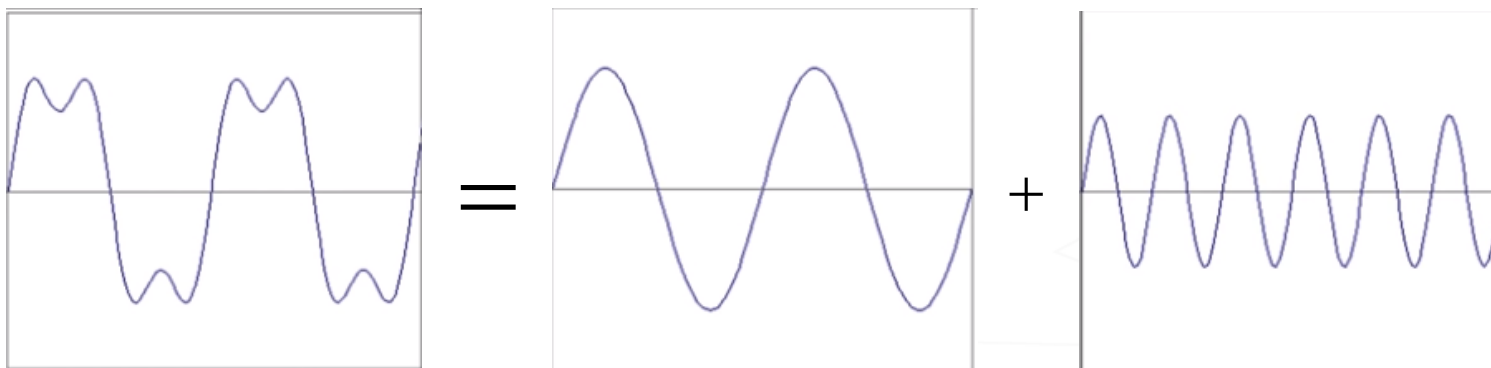
$$X_k = \sum_{n=0}^{N-1} x_n e^{-i2\pi kn/N} = \sum_{n=0}^{N-1} x_n w^{-kn} \quad k = 0, \dots, N-1$$

$$e^{ix} = \cos x + i \sin x$$

Toda función periódica puede representarse como una serie de funciones senos y cosenos.

Transformada de Fourier

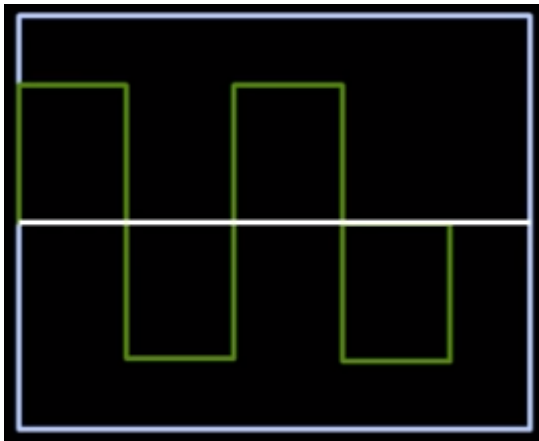
Ejemplo función casi cuadrada



Espectro de la función

Transformada de Fourier

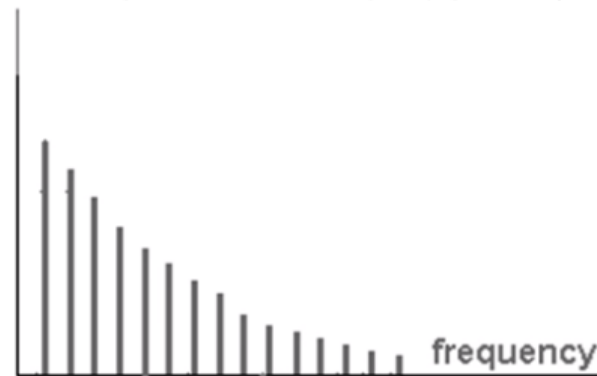
Ejemplo función cuadrada



Puede ser representada de la siguiente forma:

$$A \sum_{k=1}^{\infty} \frac{1}{k} \sin(2\pi kt)$$

Espectro de la función



Transformada de Fourier

La transformada de Fourier es entonces la descomposición de una función en todas sus componentes en frecuencia.



Conceptos a tener en cuenta:

Números complejos!

$$F(w) = R(w) + iI(w)$$

$$A = \pm \sqrt{R(w)^2 + I(w)^2}$$

$$\varphi = \tan^{-1} \frac{I(w)}{R(w)}$$

Transformada de Fourier

Definición de una transformada de Fourier (unidimensional)

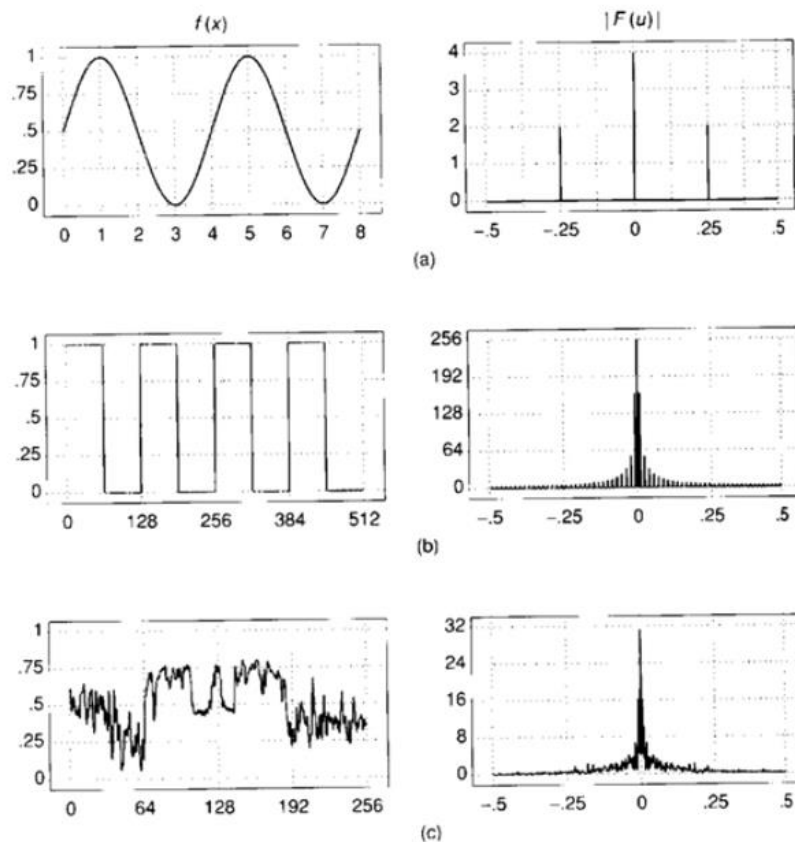
$$F(u) = \int_{-\infty}^{\infty} f(x) e^{-i2u\pi x} dx$$

Transformamos del espacio de coordenadas espaciales al espacio de frecuencias espaciales.

Definición de una transformada de Fourier inversa (unidimensional)

$$f(x) = \int_{-\infty}^{\infty} F(u) e^{i2u\pi x} du$$

Transformada de Fourier



Espectro de frecuencias para distintas funciones (unidimensionales).

Transformada de Fourier

Transformada discreta de Fourier (unidimensional)

$$F(k) = \frac{1}{N} \sum_{x=0}^{x=N-1} f(x) e^{-i \frac{2\pi kx}{N}}$$

Donde x es una variable discreta que va desde 0 hasta N-1 y k son los ciclos por periodo.

$$-N/2 < k < N/2$$

Esta es la verdadera manera en la que calculamos la transformada de Fourier en un computador!

Transformada de Fourier

Transformada de Fourier (bidimensional)

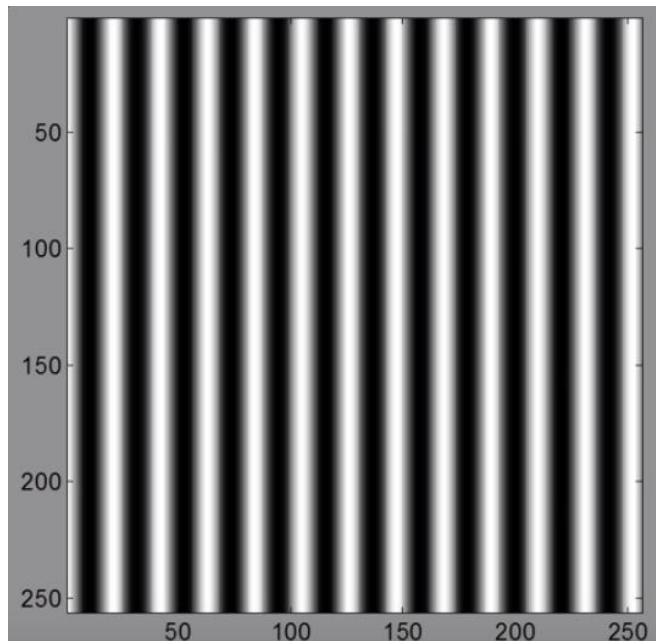
$$F(u, v) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(x, y) e^{-i2\pi(ux+vy)} dx dy$$

Transformada discreta de Fourier (bidimensional)

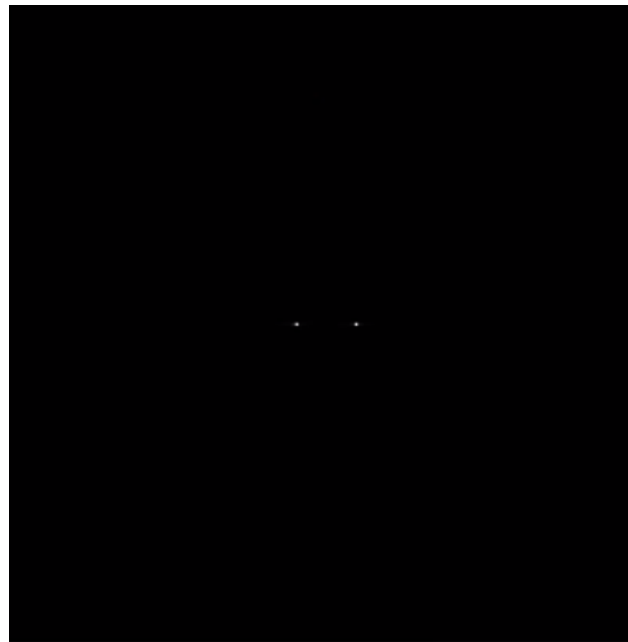
$$F(k_x, k_y) = \frac{1}{N} \sum_{x=0}^{N-1} \sum_{y=0}^{N-1} f(x, y) e^{-i \frac{2\pi(k_x x + k_y y)}{N}}$$

Transformada de Fourier

Transformada discreta de Fourier (bidimensional) **Ejemplos:**



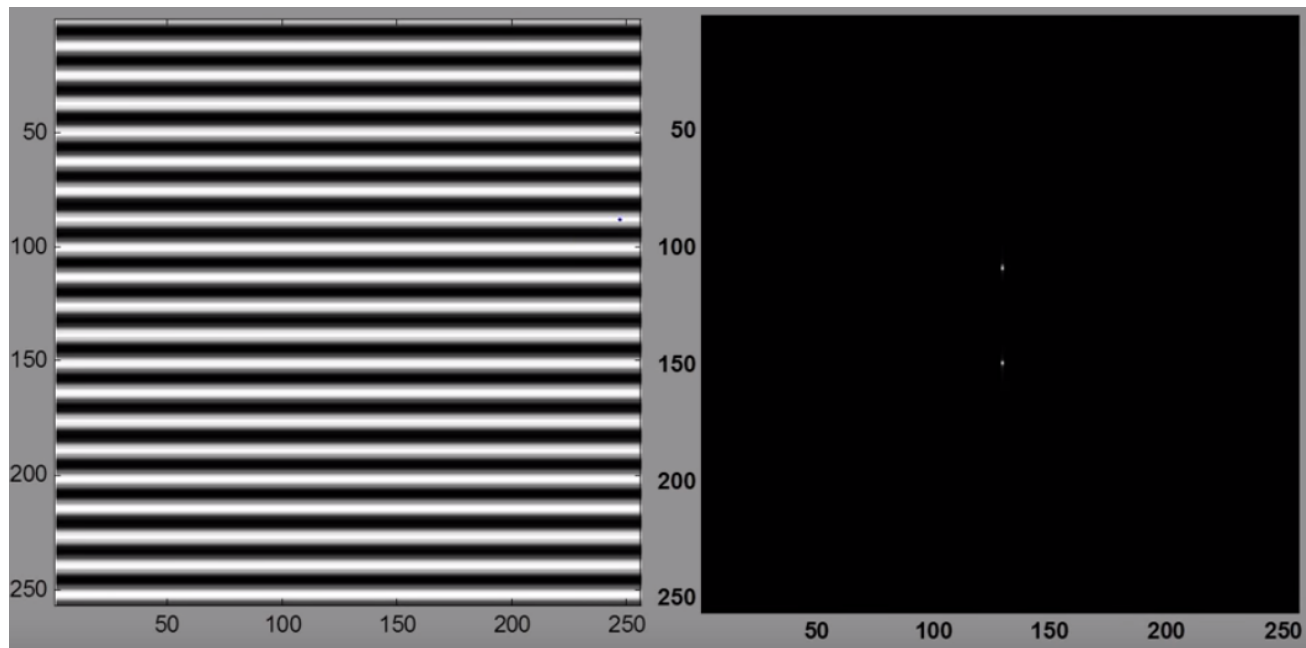
Imagen



Espectro de frecuencia

Transformada de Fourier

Transformada discreta de Fourier (bidimensional) ***Ejemplos:***

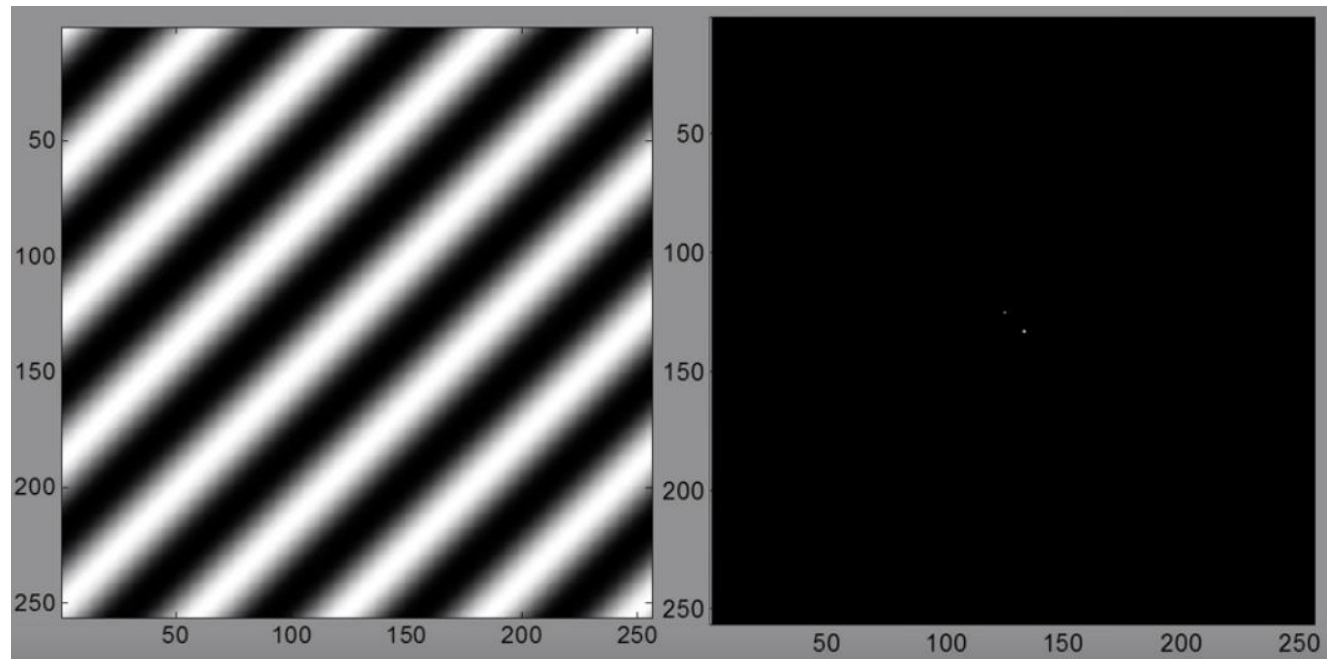


Imagen

Espectro de frecuencia

Transformada de Fourier

Transformada discreta de Fourier (bidimensional) **Ejemplos:**

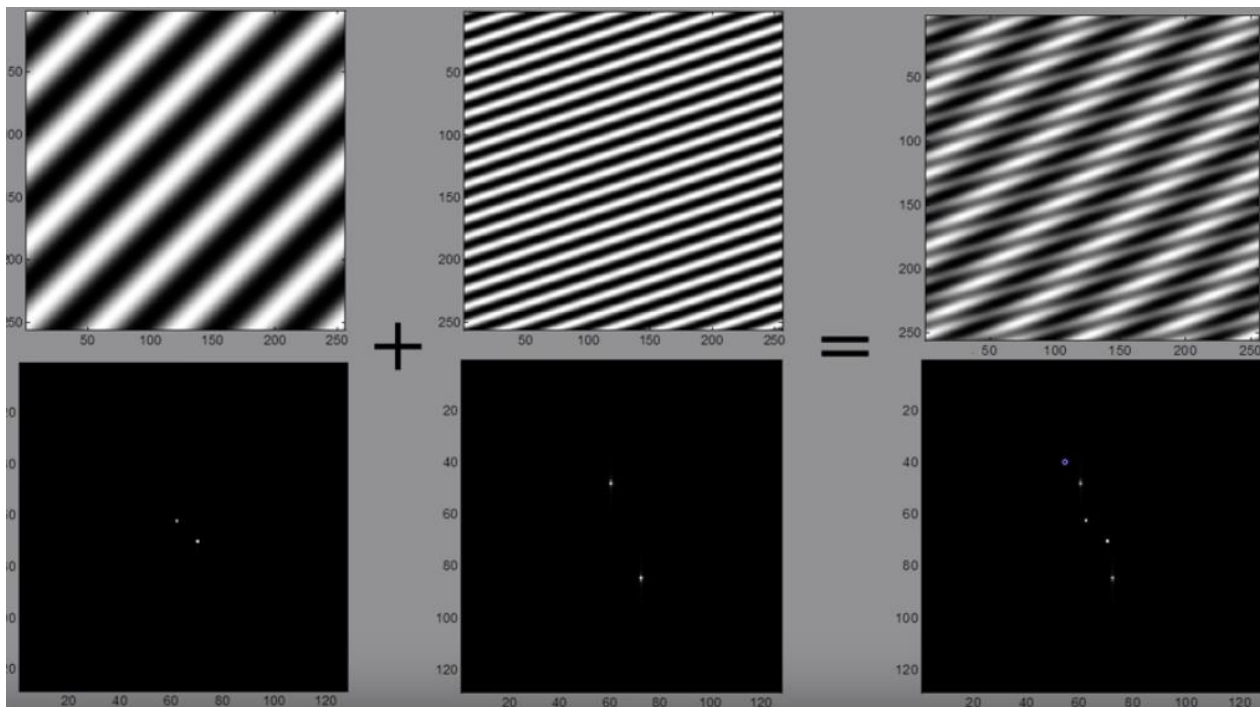


Imagen

Espectro de frecuencia

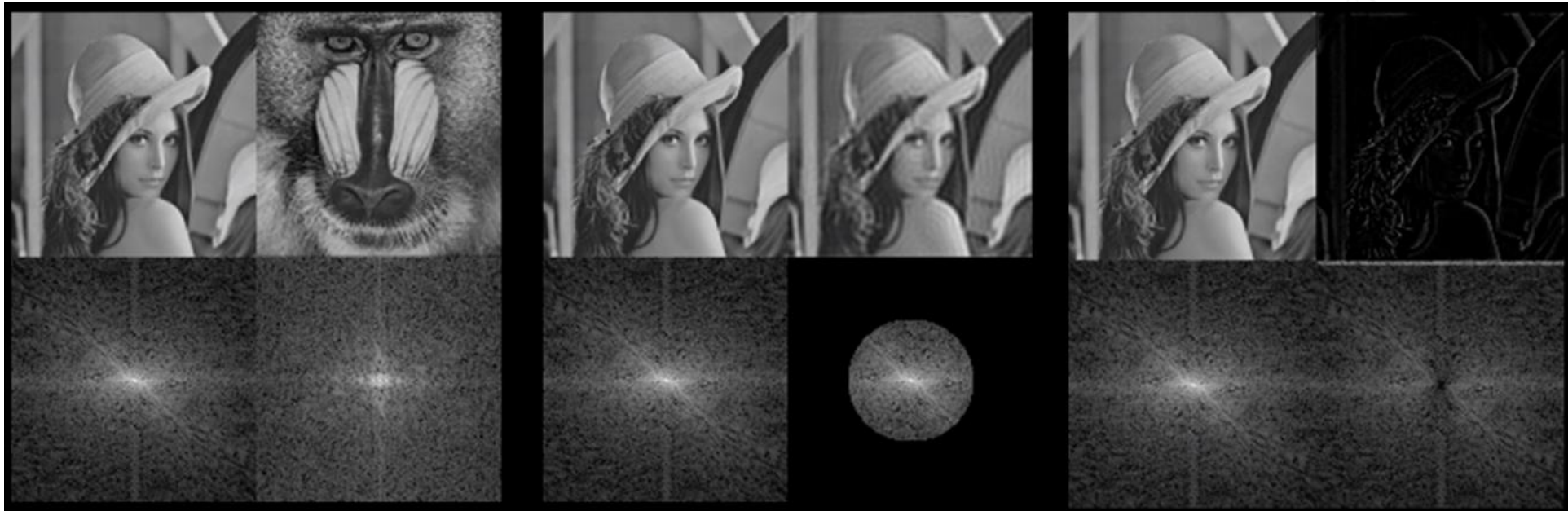
Transformada de Fourier

Transformada discreta de Fourier (bidimensional) **Linealidad**



Transformada de Fourier

Transformada discreta de Fourier (bidimensional) *Ejemplos reales.*



Filtro?

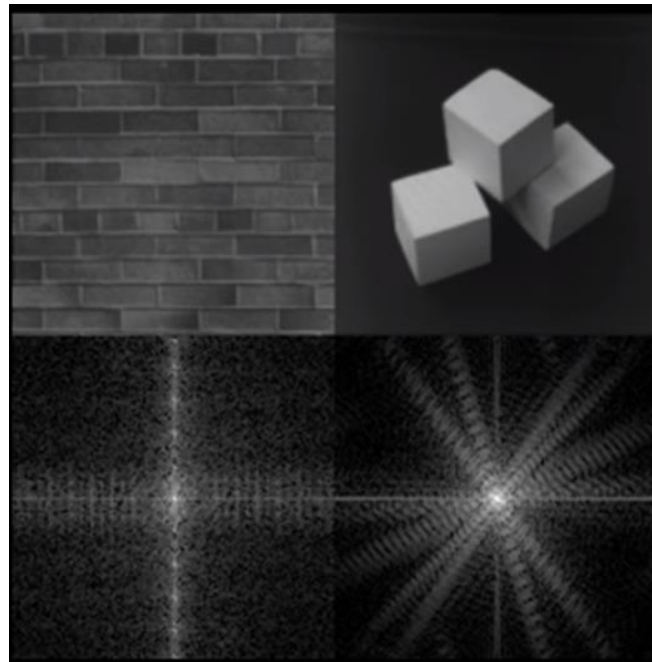
Filtro?

Transformada de Fourier

Transformada discreta de Fourier (bidimensional) ***Ejemplos reales.***



Filtro de nitidez



Transformada de Fourier

Transformada de Fourier y Convolución

$$g = f * h$$

$$G(u) = \int_{-\infty}^{\infty} g(x) e^{-i2\pi ux} dx$$

$$= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(\tau) h(x - \tau) e^{-i2\pi ux} d\tau dx$$

$$= \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} \left[f(\tau) e^{-i2\pi u\tau} d\tau \right] \left[h(x - \tau) e^{-i2\pi u(x - \tau)} dx \right]$$

$$= \int_{-\infty}^{\infty} \left[f(\tau) e^{-i2\pi u\tau} d\tau \right] \int_{-\infty}^{\infty} \left[h(x') e^{-i2\pi ux'} dx' \right]$$

$$= F(u) H(u)$$

Spatial Domain (x)		Frequency Domain (u)
$g = f * h$	$\longleftrightarrow$	$G = F \cdot H$
$g = f \cdot h$	$\longleftrightarrow$	$G = F * H$

Por consiguiente, una convolución en el dominio del espacio es una multiplicación en el dominio de la frecuencia.

Transformada de Fourier

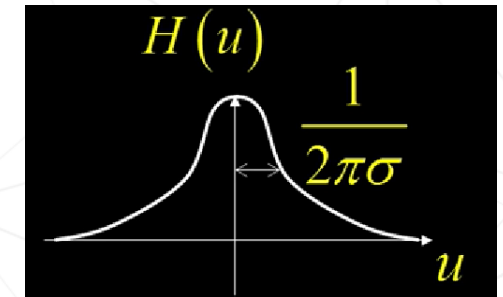
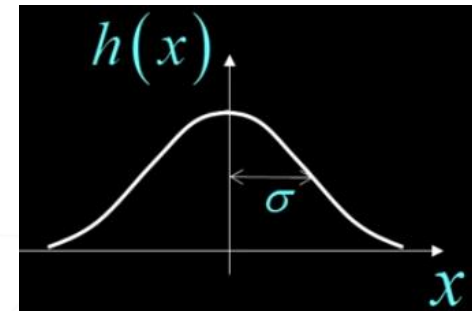
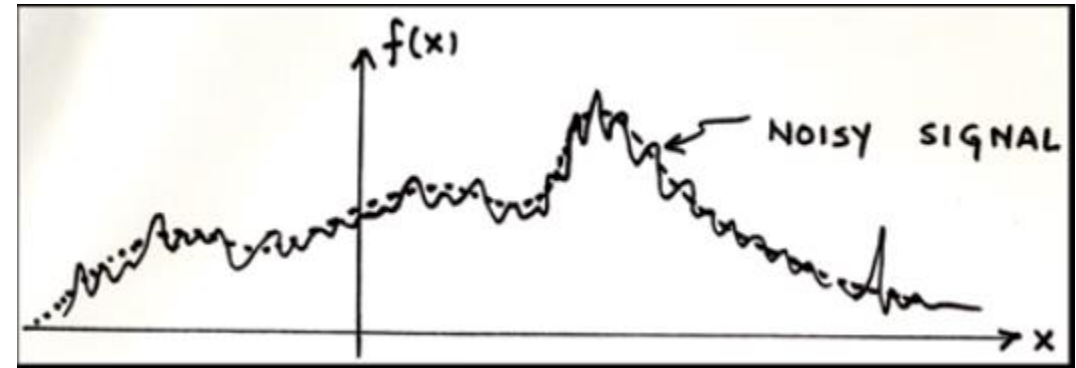
Suavizado

$$g(x) = f(x) * h(x)$$

$$h(x) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left[-\frac{1}{2} \frac{x^2}{\sigma^2}\right]$$

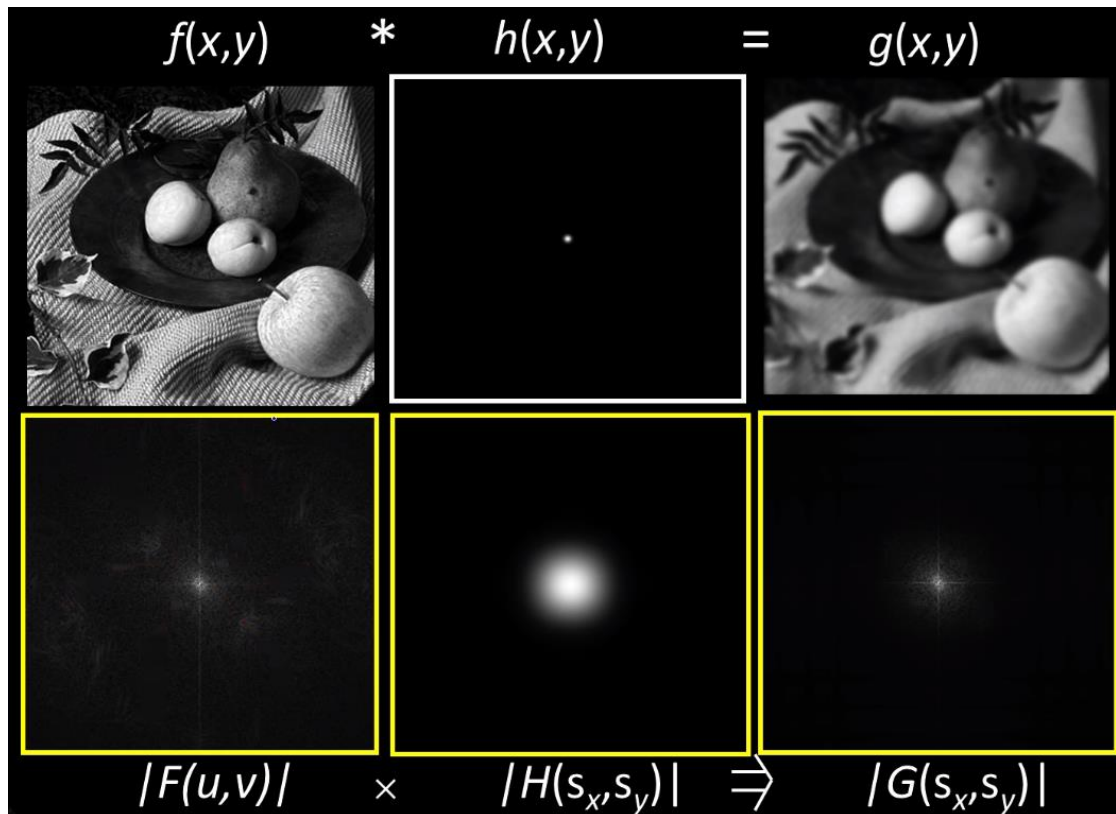
$$H(u) = \exp\left[-\frac{1}{2} (2\pi u)^2 \sigma^2\right]$$

$$G(u) = F(u)H(u)$$



Transformada de Fourier

Suavizado vía transformada de Fourier (y convolución)

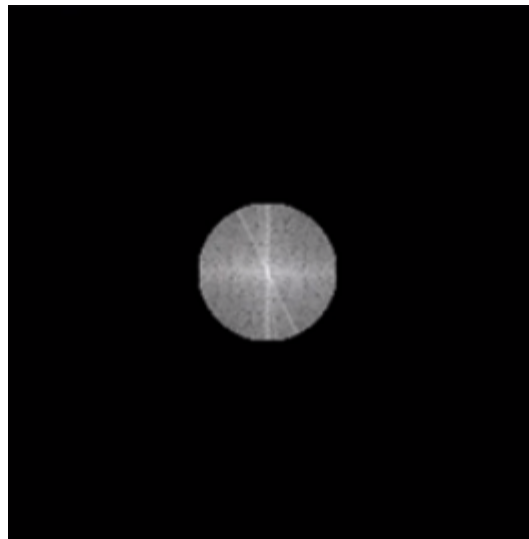


Transformada de Fourier

Filtro pasa baja con Fourier



Imagen original



Espectro más máscara



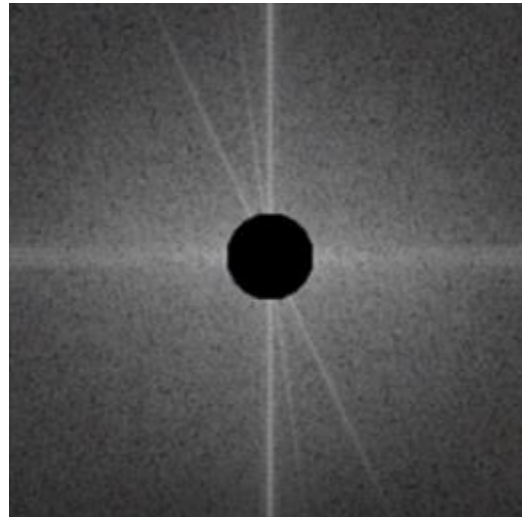
Imagen resultante después
de transformada inversa

Transformada de Fourier

Filtro pasa alto con Fourier



Imagen original



Espectro más máscara



Imagen resultante después
de transformada inversa